

3 TIER WEB APPLICATION

Project Objective:

The objective of this project is to deploy a secure, scalable, and modular 3-tier web application on AWS by integrating critical AWS services such as Elastic Load Balancing (ALB), Route 53, ACM, EC2 (frontend/backend), Amazon RDS (Aurora), Secrets Manager, IAM roles, and CloudWatch.

The web application consists of:

- **Frontend Tier:** A static HTML/JS website hosted on Apache (EC2).
- **Backend Tier:** A Flask-based REST API deployed on a separate EC2 instance.
- **Database Tier:** A MySQL-compatible Amazon Aurora instance for persistent storage.

Security and monitoring are enforced using IAM roles, Secrets Manager for secure credentials management, and CloudWatch for system-level observability.

Task 1: Connect Application Load Balancer (ALB) with Route 53

Objective:

Map a custom domain (e.g., frontend.gcpclouds.life) to the Application Load Balancer using AWS Route 53 to ensure a human-readable and reliable endpoint for accessing the app.

Steps:

i. Security Groups:

- **All_TCP:** In this security group include
 - **SSH** – Port Range: 22 – Source: Anywhere
 - **All TCP** – Port Range: 5000 – Source: Anywhere
 - **HTTP** – Port Range: 80 – Source: Anywhere

ii. Security Groups(RDS) :

- **MySQL** – Port Range: 3306 – Source: All_TCP (Internet accessed via EC2 only).

sg-0a120cc6d70b3cf6d - all_tcp

Details

Security group name all_tcp	Security group ID sg-0a120cc6d70b3cf6d	Description all_tcp	VPC ID vpc-07c7956f922556a60
Owner 767828764671	Inbound rules count 4 Permission entries	Outbound rules count 1 Permission entry	

Inbound rules (4)

Name	Security group rule ID	IP version	Type	Protocol	Port range	Source	Description
-	sgr-03e14b9ef7301a4e9	IPv4	SSH	TCP	22	0.0.0.0/0	-
-	sgr-00e0002b1490f3404	IPv4	All TCP	TCP	0 - 65535	0.0.0.0/0	-
-	sgr-0c789a4060089b096	IPv4	HTTP	TCP	80	0.0.0.0/0	-
-	sgr-00075391d762b3992	-	MYSQL/Aurora	TCP	3306	sg-0a120cc6d70b3cf6d...	-

iii. Create EC2 Instances:

- **Name:** frontend, backend, database
- **AMI Server:** Ubuntu Machine, t3.micro
- **Security Groups:** Attach the respective security groups that is created.

Instance summary for i-038e98ebab61a6045 (frontend)

Updated less than a minute ago

Instance ID i-038e98ebab61a6045	Public IPv4 address 13.204.81.245 open address	Private IPv4 addresses 172.31.7.236
IPv6 address -	Instance state Running	Public DNS ec2-13-204-81-245.ap-south-1.compute.amazonaws.com open address
Hostname type IP name: ip-172-31-7-236.ap-south-1.compute.internal	Private IP DNS name (IPv4 only) ip-172-31-7-236.ap-south-1.compute.internal	Elastic IP addresses -
Answer private resource DNS name IPv4 (A)	Instance type t2.micro	AWS Compute Optimizer finding Opt-in to AWS Compute Optimizer for recommendations. Learn more
Auto-assigned IP address 13.204.81.245 [Public IP]	VPC ID vpc-07c7956f922556a60	Auto Scaling Group name -
IAM Role -	Subnet ID subnet-084a18a2c6837df33	Managed false
IMDSv2 Required	Instance ARN arn:aws:ec2:ap-south-1:767828764671:instance/i-038e98ebab61a6045	
Operator -		

Details | Status and alarms | Monitoring | Security | Networking | Storage | Tags

Console Home | Console Home | Instance details | EC2 | ap-south-1 | SecurityGroups | EC2 | ap-south-1 | database-1 - Database Details | Students List

ap-south-1.console.aws.amazon.com/ec2/home?region=ap-south-1#InstanceDetails:instanceId=i-063063c9b9003758d

EC2 > Instances > i-063063c9b9003758d

Instance summary for i-063063c9b9003758d (backend) Info

Updated less than a minute ago

Instance ID
i-063063c9b9003758d

IPv6 address
-

Hostname type
IP name: ip-172-31-15-158.ap-south-1.compute.internal

Answer private resource DNS name
IPv4 (A)

Auto-assigned IP address
13.234.21.173 [Public IP]

IAM Role
-

IMDSv2
Required

Operator
-

Public IPv4 address
13.234.21.173 | [open address](#)

Instance state
Running

Private IP DNS name (IPv4 only)
ip-172-31-15-158.ap-south-1.compute.internal

Instance type
t2.micro

VPC ID
vpc-07c7956f922556a60

Subnet ID
subnet-084a18a2c6837df33

Instance ARN
arn:aws:ec2:ap-south-1:767828764671:instance/i-063063c9b9003758d

Private IPv4 addresses
172.31.15.158

Public DNS
ec2-13-234-21-173.ap-south-1.compute.amazonaws.com | [open address](#)

Elastic IP addresses
-

AWS Compute Optimizer finding
Opt-in to AWS Compute Optimizer for recommendations. | [Learn more](#)

Auto Scaling Group name
-

Managed
false

[Connect](#) [Instance state](#) [Actions](#)

[Details](#) [Status and alarms](#) [Monitoring](#) [Security](#) [Networking](#) [Storage](#) [Tags](#)

▼ Instance details Info

CloudShell Feedback

© 2025, Amazon Web Services, Inc. or its affiliates. Privacy Terms Cookie preferences

Console Home | Console Home | Instance details | EC2 | ap-south-1 | SecurityGroups | EC2 | ap-south-1 | database-1 - Database Details | Students List

ap-south-1.console.aws.amazon.com/ec2/home?region=ap-south-1#InstanceDetails:instanceId=i-03cdafc7fc4fb1965

EC2 > Instances > i-03cdafc7fc4fb1965

Instance summary for i-03cdafc7fc4fb1965 (database) Info

Updated less than a minute ago

Instance ID
i-03cdafc7fc4fb1965

IPv6 address
-

Hostname type
IP name: ip-172-31-2-17.ap-south-1.compute.internal

Answer private resource DNS name
IPv4 (A)

Auto-assigned IP address
3.111.39.251 [Public IP]

IAM Role
-

IMDSv2
Required

Operator
-

Public IPv4 address
3.111.39.251 | [open address](#)

Instance state
Running

Private IP DNS name (IPv4 only)
ip-172-31-2-17.ap-south-1.compute.internal

Instance type
t2.micro

VPC ID
vpc-07c7956f922556a60

Subnet ID
subnet-084a18a2c6837df33

Instance ARN
arn:aws:ec2:ap-south-1:767828764671:instance/i-03cdafc7fc4fb1965

Private IPv4 addresses
172.31.2.17

Public DNS
ec2-3-111-39-251.ap-south-1.compute.amazonaws.com | [open address](#)

Elastic IP addresses
-

AWS Compute Optimizer finding
Opt-in to AWS Compute Optimizer for recommendations. | [Learn more](#)

Auto Scaling Group name
-

Managed
false

[Connect](#) [Instance state](#) [Actions](#)

[Details](#) [Status and alarms](#) [Monitoring](#) [Security](#) [Networking](#) [Storage](#) [Tags](#)

▼ Instance details Info

CloudShell Feedback

© 2025, Amazon Web Services, Inc. or its affiliates. Privacy Terms Cookie preferences

iv. Create RDS Database:

- **Name:** database-1
- **Server:** MySQL, t3g.micro
- **Free-tier & Single AZ**
- **Set Admin username and password**
- **Database name:** 'knowledge-acquisition'

The screenshot shows the AWS RDS console for instance 'database-1'. The 'Connectivity & security' tab is active, showing the following details:

- Endpoint & port:** Endpoint is `database-1.cngac84c2in0.ap-south-1.rds.amazonaws.com`, Port is `3306`.
- Networking:** Availability Zone is `ap-south-1b`, VPC is `vpc-07c7956f922556a60`, Subnet group is `default-vpc-07c7956f922556a60`. Subnets include `subnet-084a18a2c6837df33`, `subnet-00bdabdd2ad0e63a`, and `subnet-0cbd76bd2fa91f51c`.
- Security:** VPC security groups include `all_tcp (sg-0a120cc6d70b3cf6d)` which is Active. It is not publicly accessible. Certificate authority is `rds-ca-rsa2048-g1` with a date of `May 20, 2061, 00:10 (UTC+05:30)`.

The screenshot shows the AWS RDS console for instance 'database-1'. The 'Security group rules' and 'Replication' tabs are visible.

Security group rules (3)

Security group	Type	Rule
<code>all_tcp (sg-0a120cc6d70b3cf6d)</code>	CIDR/IP - Inbound	<code>0.0.0.0/0</code>
<code>all_tcp (sg-0a120cc6d70b3cf6d)</code>	EC2 Security Group - Inbound	<code>sg-0a120cc6d70b3cf6d</code>
<code>all_tcp (sg-0a120cc6d70b3cf6d)</code>	CIDR/IP - Outbound	<code>0.0.0.0/0</code>

Replication (1)

DB identifier	Role	Region & AZ	Replication source	Replication state	Lag
<code>database-1</code>	Instance	<code>ap-south-1b</code>	-	-	-

- Then after the creation of EC2 and RDS, there is configuration need to be done in three EC2s for frontend, backend, database by giving the specific IP Address in the html files and in flask file.
- Then create a database named with 'knowledge-acquisition'
- But before that there needs a setup in all the EC2s machines(frontend, backend, database).

Frontend Instance Setup:

This instance will host your landing page, signup, login, and backend dashboard HTML/JS files.

What to Install:

Update packages

sudo apt update

Install a web server (Apache or NGINX)

sudo apt install apache2 -y

Steps:

Upload your HTML files (index.html, backend.html, student.html)

Place them inside /var/www/html/

sudo cp index.html /var/www/html/

→ index.html:

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8" />
  <meta name="viewport" content="width=device-width, initial-scale=1" />
  <title>Knowledge Acquisition</title>
  <style>
    * { box-sizing: border-box; }
    body {
      font-family: 'Segoe UI', Tahoma, Geneva, Verdana, sans-serif;
      background: #f0f4f8;
      margin: 0; padding: 0;
      display: flex; flex-direction: column;
      min-height: 100vh;
      justify-content: center; align-items: center;
      text-align: center; color: #2c3e50;
```

```
}
h1 {
  font-size: 2.5rem; margin-bottom: 30px;
}
.form-container {
  background: white;
  padding: 30px 40px;
  border-radius: 10px;
  box-shadow: 0 8px 16px rgba(0,0,0,0.1);
  width: 350px; max-width: 90%;
}
.toggle-buttons {
  margin-bottom: 20px;
  display: flex; justify-content: center; gap: 15px;
}
.toggle-buttons button {
  flex: 1;
  padding: 10px 0;
  font-size: 1rem;
  border-radius: 25px;
  border: 2px solid #3498db;
  background-color: white;
  color: #3498db;
  cursor: pointer;
  transition: all 0.3s ease;
}
.toggle-buttons button.active, .toggle-buttons button:hover {
  background-color: #3498db;
  color: white;
}
form {
  display: none;
  flex-direction: column;
  gap: 15px;
}
form.active {
  display: flex;
}
input[type="text"], input[type="email"], input[type="password"] {
  padding: 10px;
  font-size: 1rem;
  border: 1.5px solid #ccc;
  border-radius: 6px;
  outline-color: #3498db;
```

```

    transition: border-color 0.3s ease;
  }
  input[type="text"]:focus, input[type="email"]:focus, input[type="password"]:focus {
    border-color: #3498db;
  }
  button.submit-btn {
    background-color: #3498db;
    border: none;
    color: white;
    padding: 12px 0;
    font-size: 1.1rem;
    border-radius: 8px;
    cursor: pointer;
    transition: background-color 0.3s ease;
  }
  button.submit-btn:hover {
    background-color: #2980b9;
  }
  @media (max-width: 480px) {
    h1 {
      font-size: 1.8rem;
      margin-bottom: 20px;
    }
    .form-container {
      width: 90%;
      padding: 20px;
    }
    .toggle-buttons button {
      font-size: 0.9rem;
      padding: 8px 0;
    }
  }
</style>
</head>
<body>
  <h1>Welcome to Greens Tech Academic</h1>

  <div class="form-container">
    <div class="toggle-buttons">
      <button id="signin-btn" class="active">Sign In</button>
      <button id="signup-btn">Sign Up</button>
    </div>

    <!-- Sign In Form -->

```

```
<form id="signin-form" class="active" onsubmit="login(event)">
  <input type="text" id="signin-username" placeholder="Username" required />
  <input type="password" id="signin-password" placeholder="Password" required />
  <button type="submit" class="submit-btn">Sign In</button>
</form>
```

```
<!-- Sign Up Form -->
```

```
<form id="signup-form" onsubmit="signup(event)">
  <input type="text" id="signup-first_name" placeholder="First Name" required />
  <input type="text" id="signup-middle_name" placeholder="Middle Name" />
  <input type="text" id="signup-last_name" placeholder="Last Name" required />
  <input type="text" id="signup-username" placeholder="Username" required />
  <input type="password" id="signup-password" placeholder="Password" required />
  <input type="email" id="signup-email" placeholder="Email" required />
  <button type="submit" class="submit-btn">Sign Up</button>
</form>
```

```
</div>
```

```
<script>
```

```
const signinBtn = document.getElementById('signin-btn');
const signupBtn = document.getElementById('signup-btn');
const signinForm = document.getElementById('signin-form');
const signupForm = document.getElementById('signup-form');
```

```
signinBtn.addEventListener('click', () => {
  signinBtn.classList.add('active');
  signupBtn.classList.remove('active');
  signinForm.classList.add('active');
  signupForm.classList.remove('active');
});
```

```
signupBtn.addEventListener('click', () => {
  signupBtn.classList.add('active');
  signinBtn.classList.remove('active');
  signupForm.classList.add('active');
  signinForm.classList.remove('active');
});
```

```
async function signup(e) {
  e.preventDefault();
  const res = await fetch('http://<backend-public-ip>:5000/signup', {
    method: 'POST',
    headers: {'Content-Type': 'application/json'},
    body: JSON.stringify({
```



```

        first_name: document.getElementById('signup-first_name').value,
        middle_name: document.getElementById('signup-middle_name').value,
        last_name: document.getElementById('signup-last_name').value,
        username: document.getElementById('signup-username').value,
        password: document.getElementById('signup-password').value,
        email: document.getElementById('signup-email').value
    })
});
const data = await res.json();
alert(data.msg);
if(res.ok) {
    // After successful signup, switch to sign in form
    signinBtn.click();
}
}

async function login(e) {
    e.preventDefault();
    const res = await fetch('http://<backend-public-ip>:5000/login', {
        method: 'POST',
        headers: {'Content-Type': 'application/json'},
        body: JSON.stringify({
            username: document.getElementById('signin-username').value,
            password: document.getElementById('signin-password').value
        })
    });
    const data = await res.json();
    alert(data.msg);
    if(res.ok) {
        // Redirect to backend page after successful login
        window.location.href = 'backend.html';
    }
}
</script>
</body>
</html>

```

sudo cp backend.html /var/www/html/

→ backend.html:

```

<h2>Department: DevOps</h2>

<form onsubmit="saveStudent(event)" class="student-form">

    <input placeholder="Name" id="name" required />

```

```
<input type="date" id="date" required />
<input placeholder="Fees Paid" id="fees" required />
<input placeholder="Trainer Name" id="trainer" required />
<input placeholder="Company Name" id="company" required />
<input value="DevOps" id="department" readonly />
<button type="submit">Submit</button>
</form>
```

```
<style>
.student-form {
  max-width: 400px;
  margin: 0 auto 40px auto;
  background: #fff;
  padding: 25px 30px;
  border-radius: 10px;
  box-shadow: 0 8px 16px rgba(0,0,0,0.1);
  display: flex;
  flex-direction: column;
  gap: 15px;
  font-family: 'Segoe UI', Tahoma, Geneva, Verdana, sans-serif;
}

.student-form input {
  padding: 10px 15px;
  font-size: 1rem;
  border: 1.5px solid #ccc;
  border-radius: 6px;
  outline-color: #3498db;
  transition: border-color 0.3s ease;
}
```

```
.student-form input:focus {  
  border-color: #3498db;  
}  
  
.student-form input[readonly] {  
  background-color: #e9ecef;  
  cursor: not-allowed;  
}
```

```
.student-form button {  
  padding: 12px 0;  
  font-size: 1.1rem;  
  border: none;  
  border-radius: 8px;  
  background-color: #3498db;  
  color: white;  
  cursor: pointer;  
  transition: background-color 0.3s ease;  
}
```

```
.student-form button:hover {  
  background-color: #2980b9;  
}
```

```
h2 {  
  text-align: center;  
  font-family: 'Segoe UI', Tahoma, Geneva, Verdana, sans-serif;  
  margin-bottom: 20px;  
  color: #2c3e50;  
}
```

```
@media (max-width: 480px) {
```

```

.student-form {
  padding: 20px;
  max-width: 90%;
  gap: 12px;
}
.student-form input, .student-form button {
  font-size: 1rem;
  padding: 10px 12px;
}
}
</style>

<script>
async function saveStudent(e) {
  e.preventDefault();
  const res = await fetch('http://<backend-public-ip>:5000/student', {
    method: 'POST',
    headers: {'Content-Type': 'application/json'},
    body: JSON.stringify({
      name: document.getElementById('name').value,
      date_of_joining: document.getElementById('date').value,
      fees_paid: document.getElementById('fees').value,
      department: document.getElementById('department').value,
      trainer_name: document.getElementById('trainer').value,
      company_name: document.getElementById('company').value
    })
  });

  const data = await res.json();
  alert(data.msg);
}

```

```
if (res.ok) {  
    window.location.href = 'student.html';  
}  
}  
</script>
```

sudo cp student.html /var/www/html/

→ student.html:

```
<!DOCTYPE html>  
<html lang="en">  
<head>  
    <meta charset="UTF-8" />  
    <title>Students List</title>  
    <style>  
        body {  
            font-family: 'Segoe UI', Tahoma, Geneva, Verdana, sans-serif;  
            max-width: 900px;  
            margin: 40px auto;  
            padding: 10px;  
            background: #f9f9f9;  
        }  
        h2 {  
            text-align: center;  
            color: #2c3e50;  
        }  
        table {  
            width: 100%;  
            border-collapse: collapse;  
            margin-top: 20px;  
            box-shadow: 0 4px 12px rgba(0,0,0,0.1);  
        }  
        th, td {  
            padding: 12px;  
            border: 1px solid #ddd;  
            text-align: left;  
        }  
        th {  
            background-color: #3498db;  
            color: white;  
        }  
    </style>  
</head>  
<body>  
    <h2>Students List</h2>  
    <table>  
        <tr>  
            <th>ID</th>  
            <th>Name</th>  
            <th>Email</th>  
            <th>Phone</th>  
            <th>Address</th>  
        </tr>  
        <tr>  
            <td>1</td>  
            <td>John Doe</td>  
            <td>john.doe@example.com</td>  
            <td>1234567890</td>  
            <td>123 Main St, New York, NY 10001</td>  
        </tr>  
        <tr>  
            <td>2</td>  
            <td>Jane Smith</td>  
            <td>jane.smith@example.com</td>  
            <td>9876543210</td>  
            <td>456 Elm St, Los Angeles, CA 90001</td>  
        </tr>  
        <tr>  
            <td>3</td>  
            <td>Bob Johnson</td>  
            <td>bob.johnson@example.com</td>  
            <td>0987654321</td>  
            <td>789 Oak St, Chicago, IL 60601</td>  
        </tr>  
        <tr>  
            <td>4</td>  
            <td>Alice Brown</td>  
            <td>alice.brown@example.com</td>  
            <td>2109876543</td>  
            <td>321 Pine St, San Francisco, CA 94101</td>  
        </tr>  
        <tr>  
            <td>5</td>  
            <td>Charlie Davis</td>  
            <td>charlie.davis@example.com</td>  
            <td>5432109876</td>  
            <td>654 Maple St, Austin, TX 78701</td>  
        </tr>  
    </table>  
</body>  
</html>
```

```

tr:nth-child(even) {
    background-color: #f2f2f2;
}
@media (max-width: 600px) {
    table, thead, tbody, th, td, tr {
        display: block;
    }
    th {
        position: absolute;
        top: -9999px;
        left: -9999px;
    }
    tr {
        margin-bottom: 15px;
    }
    td {
        border: none;
        position: relative;
        padding-left: 50%;
        text-align: right;
    }
    td::before {
        position: absolute;
        left: 15px;
        width: 45%;
        white-space: nowrap;
        content: attr(data-label);
        font-weight: bold;
        text-align: left;
    }
}
</style>
</head>
<body>

```

```
<h2>Students Registered</h2>
```

```
<table id="students-table">
```

```
<thead>
```

```
<tr>
```

```
<th>Name</th>
```

```
<th>Date of Joining</th>
```

```
<th>Fees Paid</th>
```

```
<th>Department</th>
```

```
<th>Trainer Name</th>
```

```

    <th>Company Name</th>
  </tr>
</thead>
<tbody>
  <!-- Students data will be inserted here -->
</tbody>
</table>

<script>
  async function loadStudents() {
    try {
      const res = await fetch('http://<backend-public-ip>:5000/students'); // Your backend
should expose this API
      if (!res.ok) throw new Error('Failed to fetch students');
      const students = await res.json();

      const tbody = document.querySelector('#students-table tbody');
      tbody.innerHTML = ""; // Clear existing rows

      students.forEach(student => {
        const tr = document.createElement('tr');

        tr.innerHTML = `
          <td data-label="Name">${student.name}</td>
          <td data-label="Date of Joining">${new
Date(student.date_of_joining).toLocaleDateString()}</td>
          <td data-label="Fees Paid">${student.fees_paid}</td>
          <td data-label="Department">${student.department}</td>
          <td data-label="Trainer Name">${student.trainer_name}</td>
          <td data-label="Company Name">${student.company_name}</td>
        `;

        tbody.appendChild(tr);
      });
    } catch (error) {
      console.error('Error loading students:', error);
      alert('Error loading students. Check console.');
```

</html>

Start Apache

```
sudo systemctl start apache2  
sudo systemctl enable apache2
```

Allow HTTP in firewall (if using UFW)

```
sudo ufw allow 'Apache'
```

Access your landing page

Open a browser and enter: `http://<frontend-public-ip>/`

Backend Instance Setup:

This instance will host your Flask REST API (signup, login, save student info).

What to Install

```
# Update & install Python3 and pip  
sudo apt update  
sudo apt install python3 python3-pip -y  
sudo apt install python3-venv -y
```

2. Create a virtual environment

```
python3 -m venv myenv
```

3. Activate the virtual environment

```
source myenv/bin/activate
```

You will now see your shell change to something like:

```
(myenv) root@ip-172-31-40-113:~#
```

4. Now install your packages safely

```
pip install mysql-connector-python #(Backend Command)  
pip install flask-cors #(Backend Command)  
pip install flask flask-mysql flask-cors bcrypt
```

→ app.py:

```
from flask import Flask, request, jsonify  
from flask_cors import CORS
```



```
import mysql.connector
```

```
import bcrypt
```

```
app = Flask(__name__)
```

```
CORS(app)
```

```
conn = mysql.connector.connect(  
    host='db-endpoint',  
    user='admin',  
    password='Prabath123',  
    database='knowledge_acquisition'  
)
```

```
cursor = conn.cursor()
```

```
@app.route('/signup', methods=['POST'])
```

```
def signup():
```

```
    data = request.json
```

```
    hashed_pw = bcrypt.hashpw(data['password'].encode('utf-8'), bcrypt.gensalt())
```

```
    cursor.execute("INSERT INTO users (first_name, middle_name, last_name,  
username, password, email) VALUES (%s, %s, %s, %s, %s, %s)",
```

```
        (data['first_name'], data['middle_name'], data['last_name'],  
data['username'], hashed_pw, data['email']))
```

```
    conn.commit()
```

```
    return jsonify({'msg': 'User registered successfully'})
```

```
@app.route('/login', methods=['POST'])
```

```
def login():
```

```
    data = request.json
```

```
    cursor.execute("SELECT password FROM users WHERE username = %s",  
(data['username'],))
```

```
    result = cursor.fetchone()
```

```
if result and bcrypt.checkpw(data['password'].encode('utf-8'), result[0].encode('utf-8')):
```

```
    return jsonify({'msg': 'Login successful'})
```

```
    return jsonify({'msg': 'Invalid credentials'}), 401
```

```
@app.route('/')
```

```
def home():
```

```
    return 'Welcome to the Knowledge Acquisition API'
```

```
@app.route('/student', methods=['POST'])
```

```
def student():
```

```
    data = request.json
```

```
    cursor = conn.cursor()
```

```
    cursor.execute("INSERT INTO students (name, date_of_joining, fees_paid, department, trainer_name, company_name) VALUES (%s, %s, %s, %s, %s, %s)",
```

```
                    (data['name'], data['date_of_joining'], data['fees_paid'], data['department'], data['trainer_name'], data['company_name']))
```

```
    conn.commit()
```

```
    cursor.close()
```

```
    return jsonify({'msg': 'Student data stored'})
```

```
@app.route('/students', methods=['GET'])
```

```
def get_students():
```

```
    cursor = conn.cursor(dictionary=True)
```

```
    cursor.execute("SELECT * FROM students") # Use your actual students table name
```

```
    students = cursor.fetchall()
```

```
    cursor.close()
```

```
    return jsonify(students)
```

```
if __name__ == '__main__':  
    app.run(host='0.0.0.0', port=5000)
```

These will be installed only inside myenv and won't affect your system-wide Python.

Every time you reboot or open a new terminal:

You need to activate the virtual environment again before running your Flask app:
source myenv/bin/activate

=====

DataBase Instance Setup:

```
sudo apt update  
sudo apt install mysql-server -y  
sudo systemctl start mysql  
sudo mysql_secure_installation
```

```
mysql -h <endpoint> -P 3306 -u admin -p
```

→ MySQL code:

```
CREATE DATABASE knowledge_acquisition;  
USE knowledge_acquisition;
```

```
CREATE TABLE users (  
    id INT AUTO_INCREMENT PRIMARY KEY,  
    first_name VARCHAR(100),  
    middle_name VARCHAR(100),  
    last_name VARCHAR(100),  
    username VARCHAR(100) UNIQUE,  
    password VARCHAR(255),  
    email VARCHAR(100)  
);
```

```
CREATE TABLE students (  
    id INT AUTO_INCREMENT PRIMARY KEY,  
    name VARCHAR(255),  
    date_of_joining DATE,  
    fees_paid DECIMAL(10,2),
```

```
department VARCHAR(100),
trainer_name VARCHAR(100),
company_name VARCHAR(100)
);
```

➤ The following Codes need to be imported and must be changed in the specific lines.

v. Create Target Group

- **Name:** frontend-tg or backend-tg.
- **Target Type:** Instance
- **Protocol:** HTTP
- **Port:** 80 for frontend / 5000 for backend
- **Health check:** Path / or /index.html

The screenshot displays the AWS Management Console interface for a Target Group named 'frontend-tg'. The console shows the following details:

- Target type:** Instance
- IP address type:** IPv4
- Protocol : Port:** HTTP: 80
- Load balancer:** frontend-alb
- Protocol version:** HTTP1
- VPC:** vpc-07c7956f922556a60

Below the details, there is a summary of targets:

Target ID	Health	Unhealthy	Unused	Initial	Draining
1	1 Healthy	0 Unhealthy	0 Unused	0 Initial	0 Draining

The 'Registered targets' section shows a table with the following data:

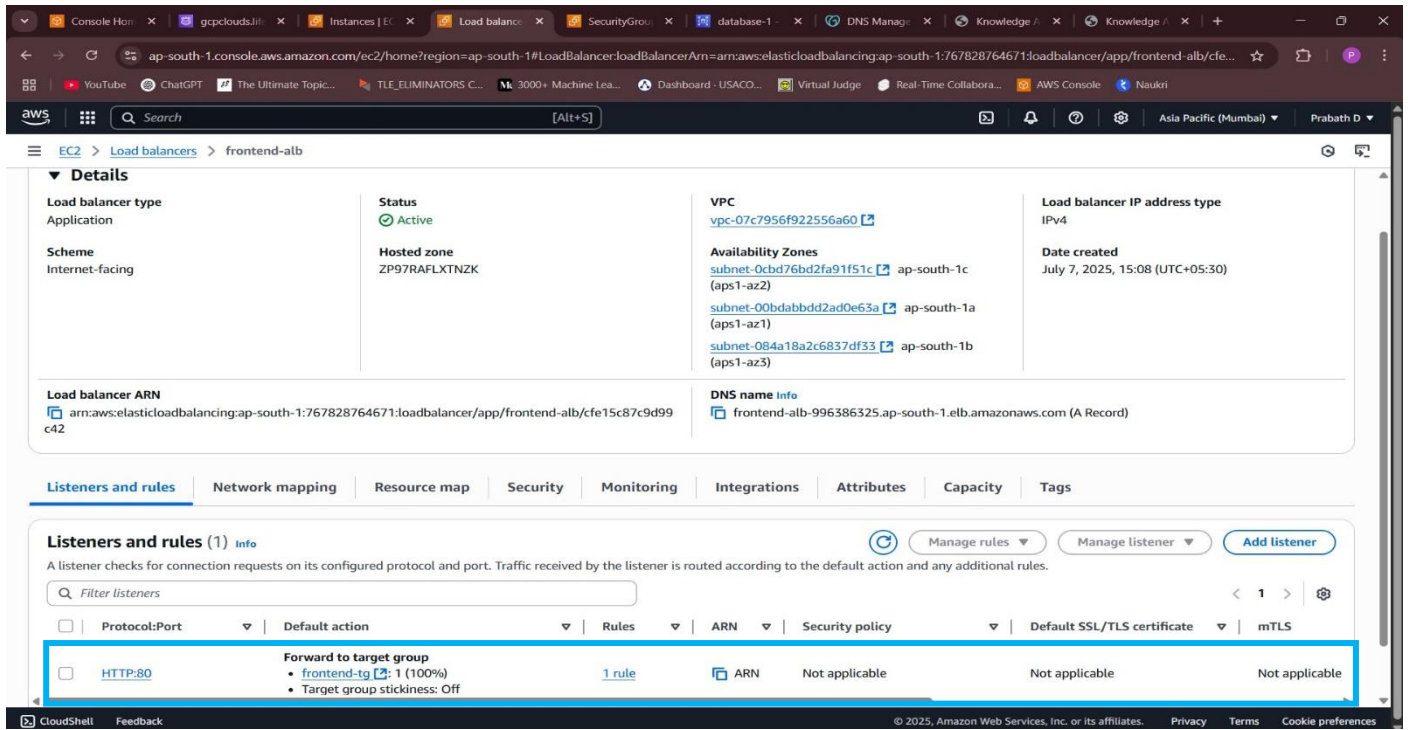
Instance ID	Name	Port	Zone	Health status	Health status details	Admini...	Overri...	Launch...	Anon...
i-038e98ebab61a6045	frontend	80	ap-south-1b (a...	Healthy	-	No override	No overri...	July 7, 20...	Green

vi. Create ALB

- **Name:** frontend-alb or backend-alb
- **Scheme:** Internet-facing
- **Protocols:** Add listeners for HTTP (80) and HTTPS (443)
- **Target Group:** Attach the one created above
- **Security Group:** Allow HTTP/HTTPS access

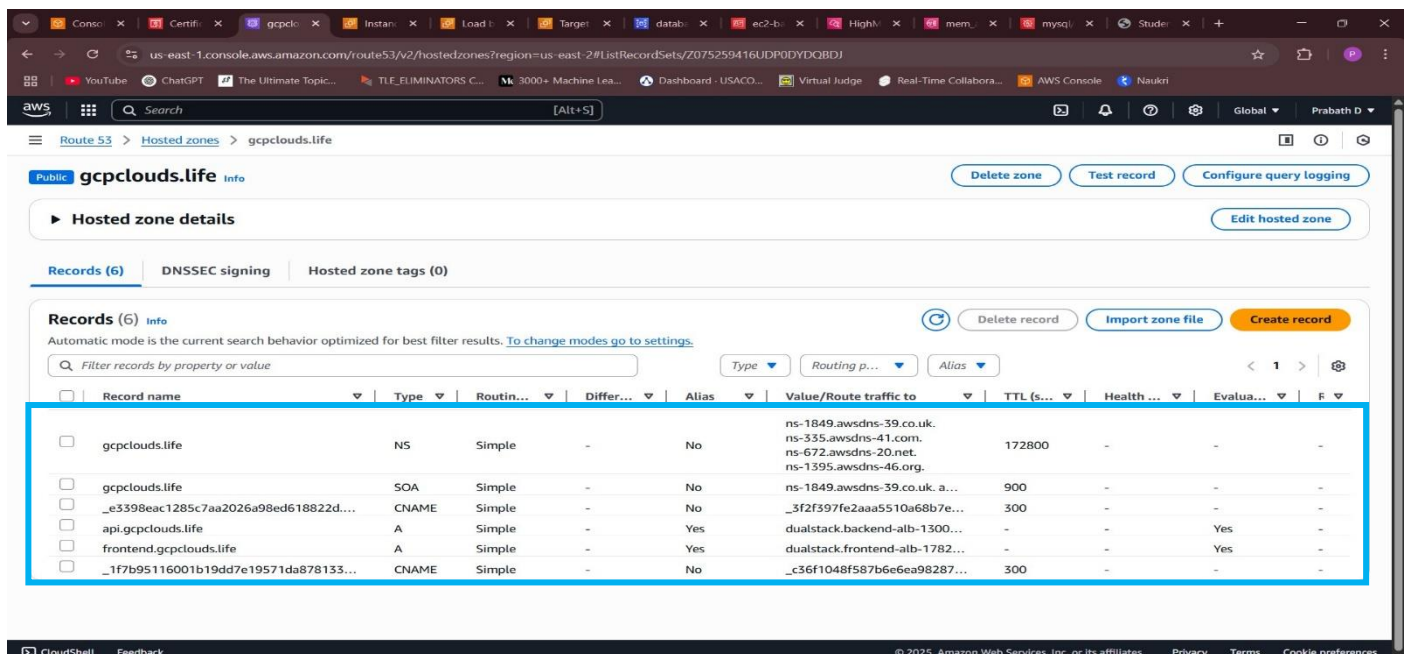
vii. Register EC2 Targets

- Add corresponding EC2 instance(s) to the ALB target group.
- Ensure the EC2 health checks pass.



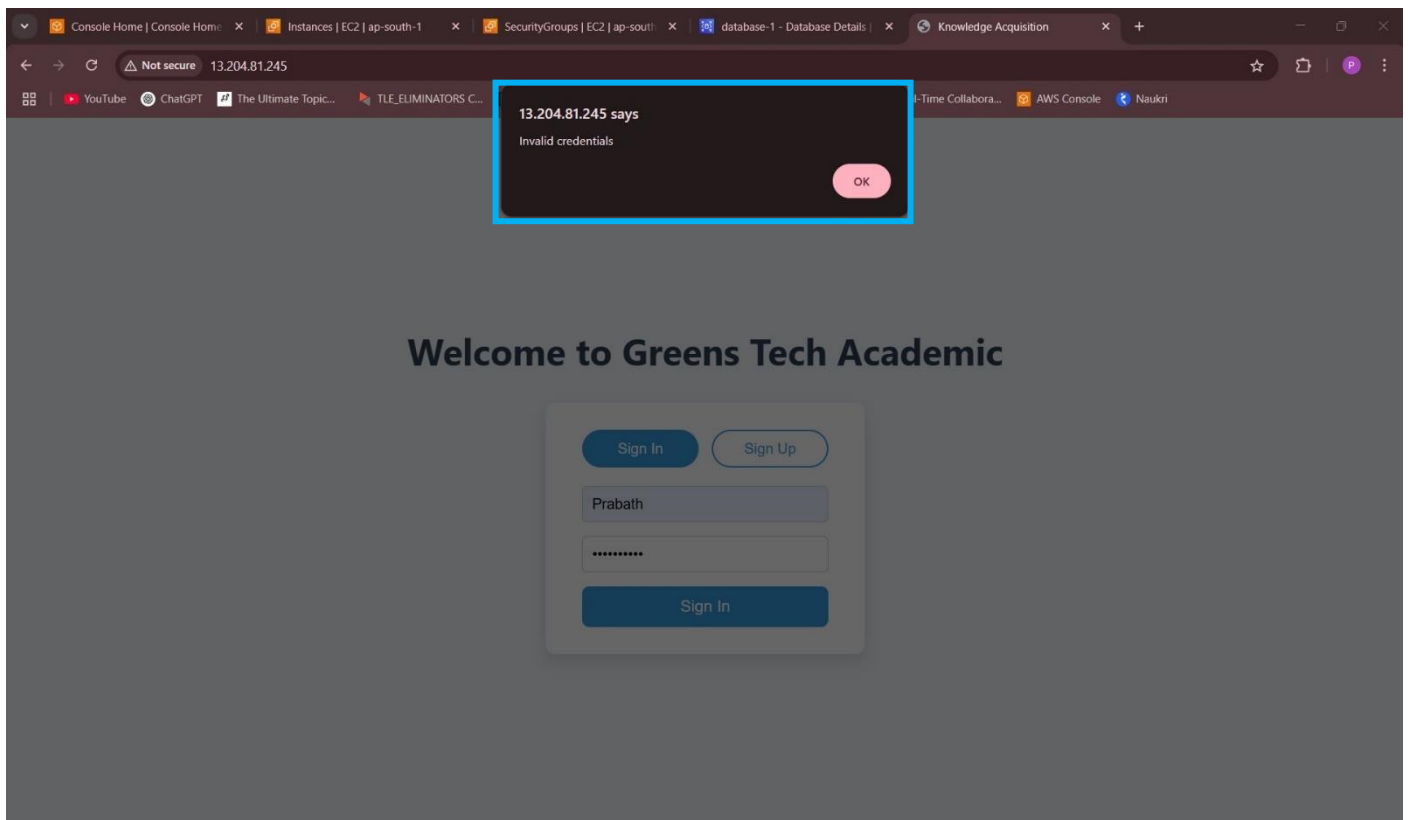
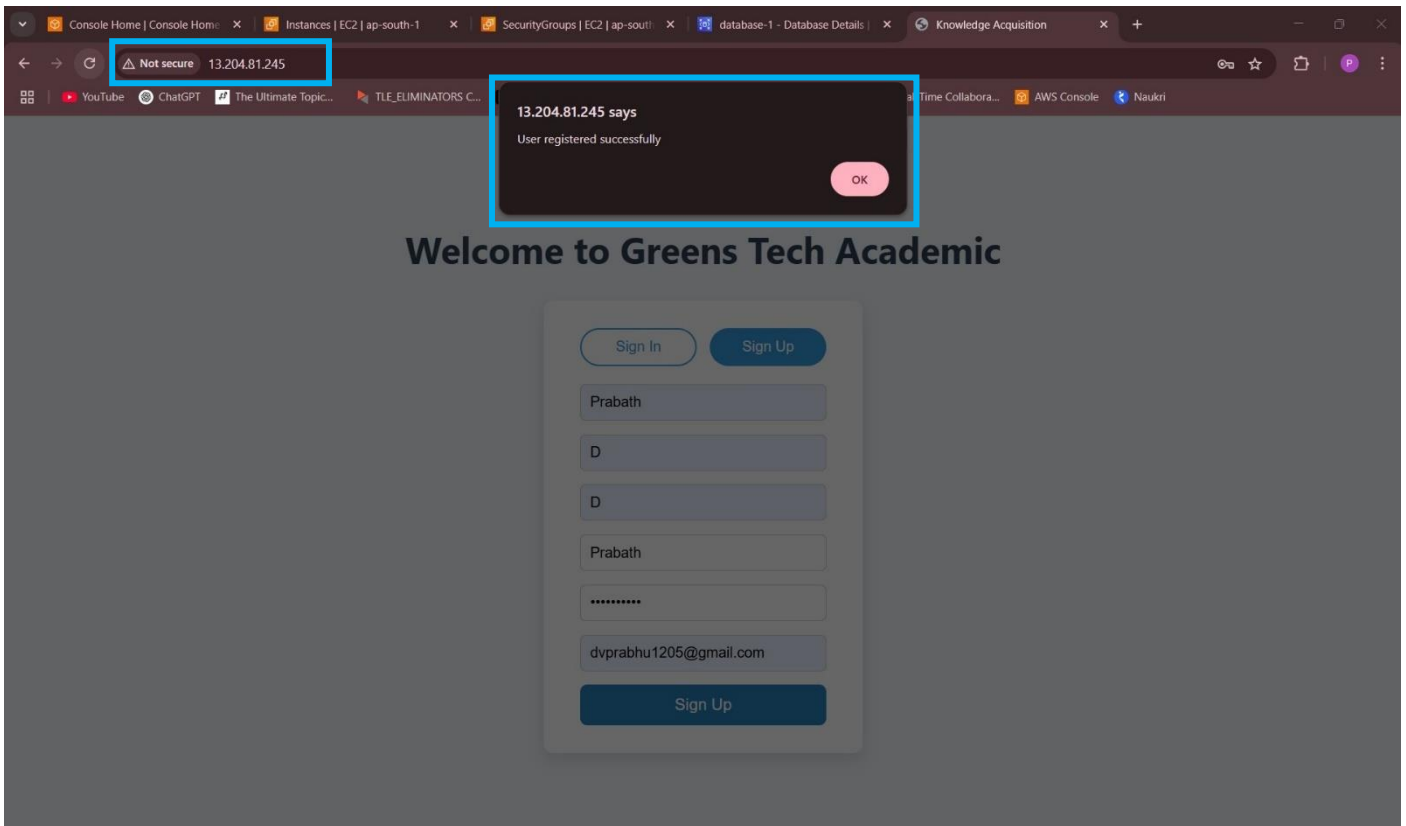
viii. Route 53 Configuration

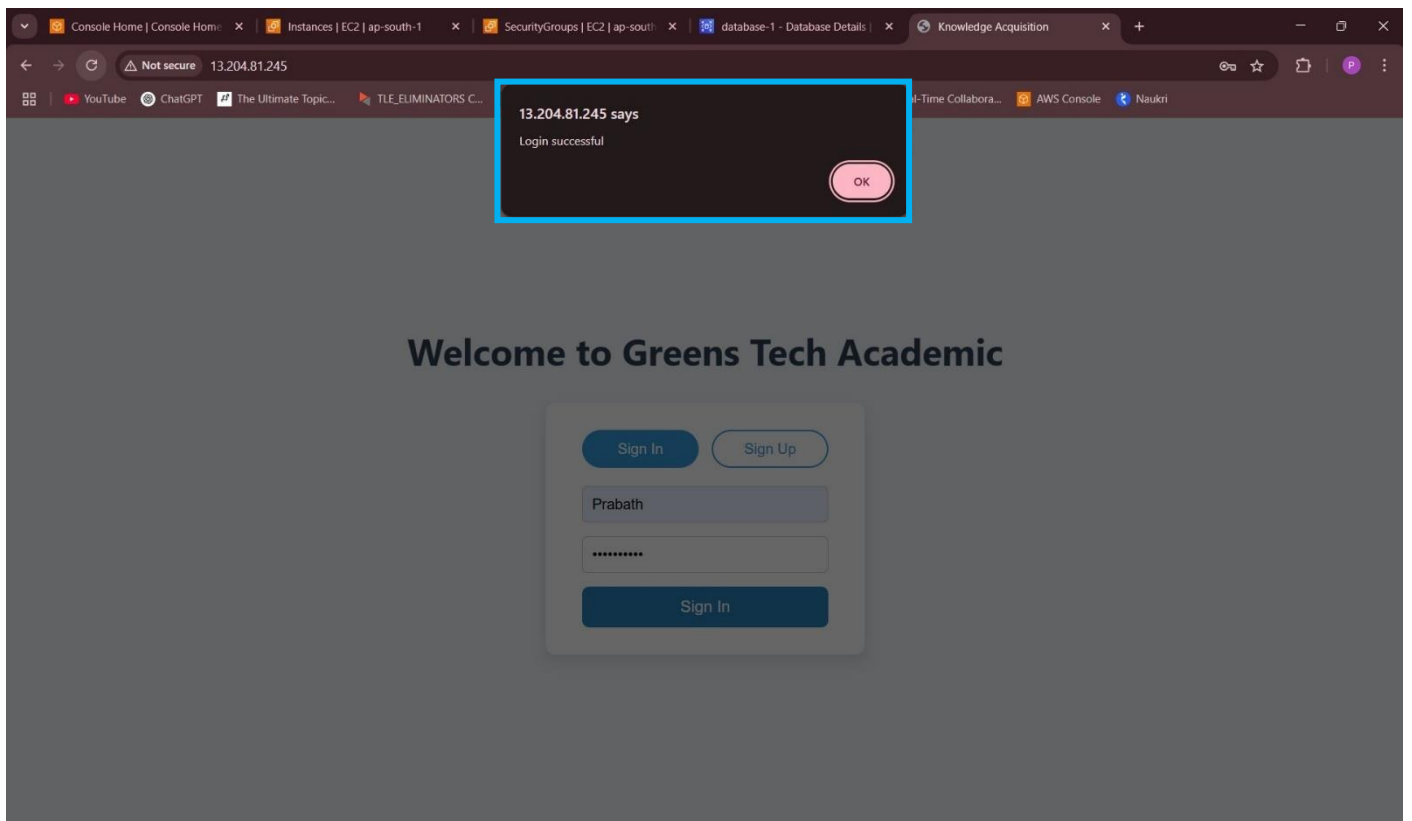
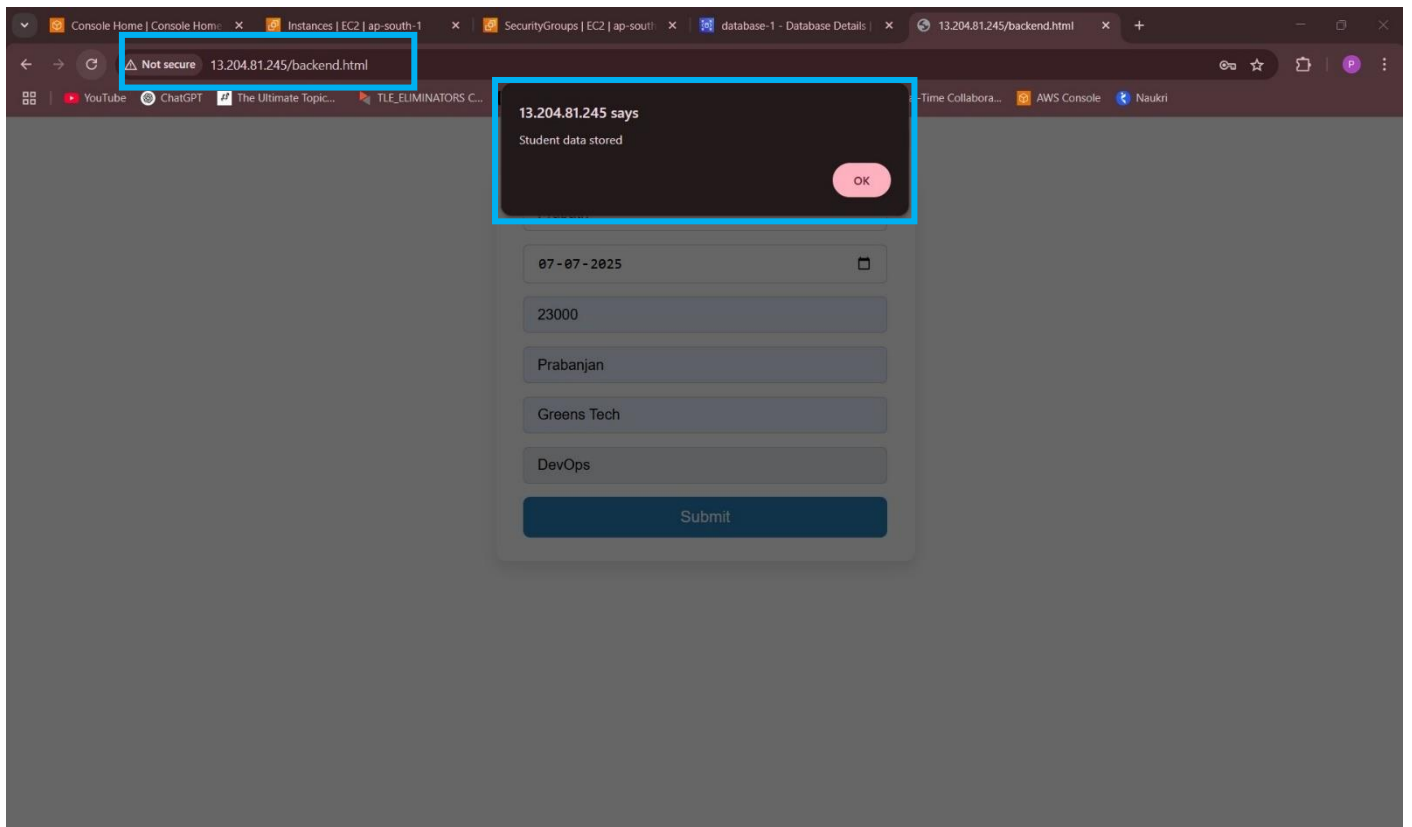
- Create a hosted zone: gpcclouds.life
- Create an A Record (Alias: Yes)
 - Name: frontend.gpcclouds.life → ALB DNS name
 - Name: api.gpcclouds.life → Backend ALB



ix. Validation

- Access the domain name in a browser to confirm successful redirection to the ALB.

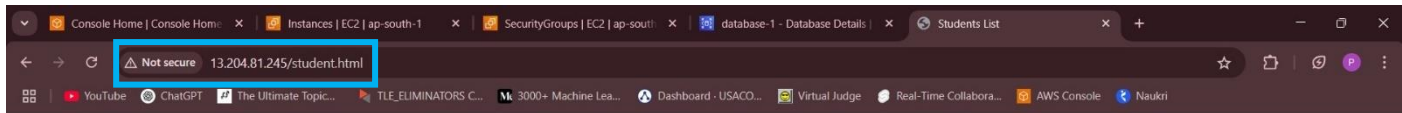




```
root@ip-172-31-2-17: ~
mysql> select * from users;
+-----+-----+-----+-----+-----+-----+
| id | first_name | middle_name | last_name | username | password |
+-----+-----+-----+-----+-----+-----+
| 1 | Local | | Test | localuser | $2b$12$XW2IgI4GRacL3LF13oJ36.zqn |
| 2 | Prabath | D | | Prabhu | $2b$12$hDrEwaF1NkVKEumZyq5c.wpC |
| 3 | Prabath | D | | Prabath | $2b$12$QhZk7Jrml2/b8qdsGumi0eqGx |
| 5 | Arun | D | A | Arun | $2b$12$xaXTWG1lWUq10/z06a67KOZ8. |
| 6 | Neeraj | T | T | Neeraj | $2b$12$XjYswWt/36nuAluSIz9NSenbj |
| 15 | Neeraj | T | T | Neeraj T | $2b$12$UYT30Gf/bKrkxz5HvvHSFeko2 |
| 16 | Darius | Rijin | I | Darius Rijin | $2b$12$FtUFlxoL8AmBZkPAsI9/gOeOe |
| 17 | Jayanthan | M | M | Jayanthan | $2b$12$HPEfbU.tDToxy0mjEzk8x.fg7 |
+-----+-----+-----+-----+-----+-----+
8 rows in set (0.00 sec)

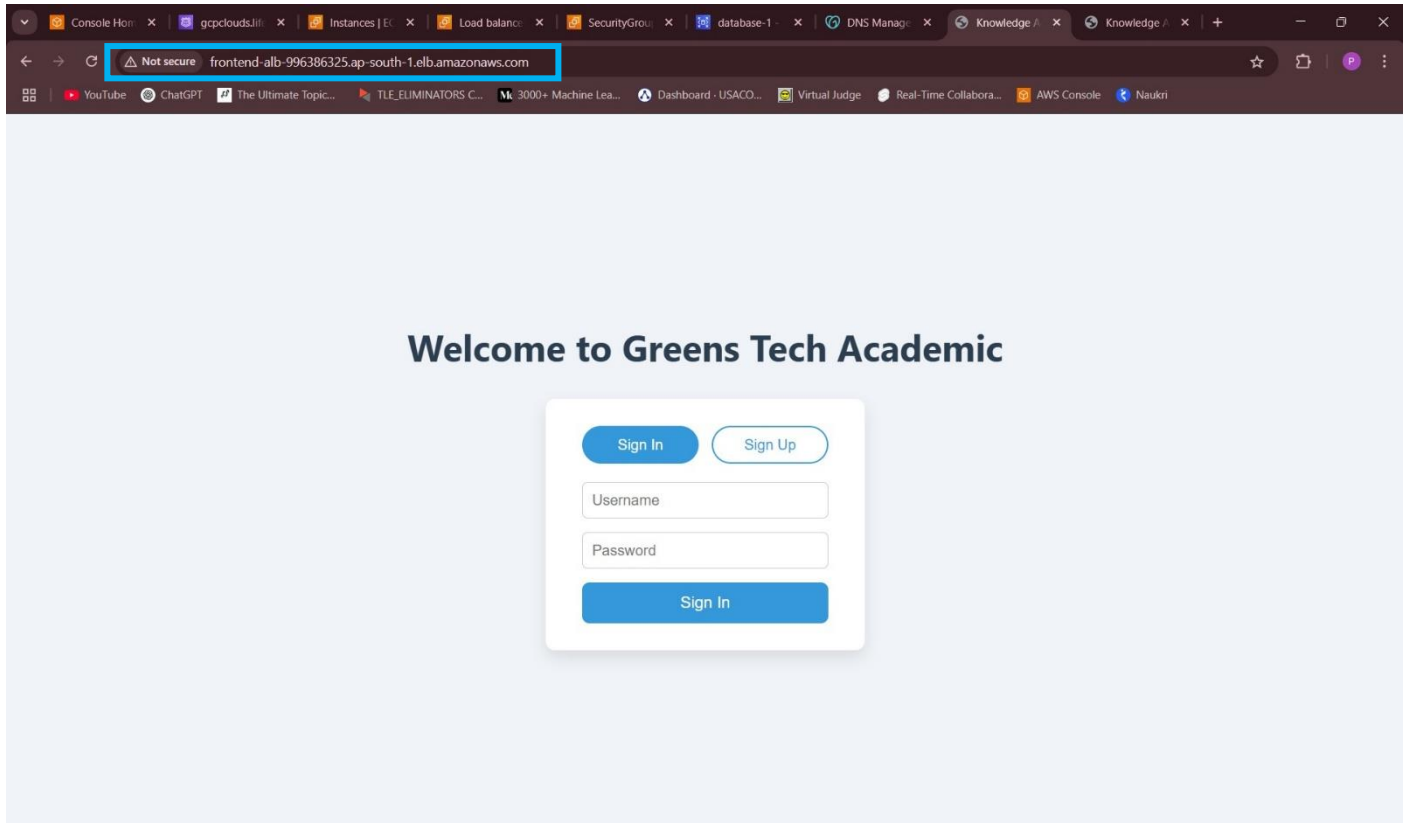
mysql> select * from students;
+-----+-----+-----+-----+-----+-----+-----+
| id | name | date_of_joining | fees_paid | department | trainer_name | company_name |
+-----+-----+-----+-----+-----+-----+-----+
| 2 | Prabath | 2025-07-07 | 23000.00 | DevOps | Prabanjan | Greens Tech |
| 3 | Arun D A | 2025-07-02 | 25000.00 | DevOps | Prabanjan | Greens Tech |
| 4 | Neeraj T | 2025-06-30 | 30000.00 | DevOps | Venkat | Greens Tech |
| 5 | Darius Rijin | 2025-07-11 | 26000.00 | DevOps | Pavan | Greens Tech |
| 6 | Jayanthan | 2025-07-04 | 20000.00 | DevOps | Mubarak | Greens Tech |
+-----+-----+-----+-----+-----+-----+-----+
```

Students stored in MySQL



Students Registered

Name	Date of Joining	Fees Paid	Department	Trainer Name	Company Name
Prabath	7/7/2025	23000.00	DevOps	Prabanjan	Greens Tech
Arun D A	7/2/2025	25000.00	DevOps	Prabanjan	Greens Tech
Neeraj T	6/30/2025	30000.00	DevOps	Venkat	Greens Tech
Darius Rijin	7/11/2025	26000.00	DevOps	Pavan	Greens Tech
Jayanthan	7/4/2025	20000.00	DevOps	Mubarak	Greens Tech



Hosted through DNS from created LB

- **Henceforth, the task to implement Connect Application Load Balancer (ALB) with Route 53 is completed.**

Task 2: Enable HTTPS using ACM and ALB Listener Rules

Objective:

Secure application communication using an SSL/TLS certificate issued via AWS Certificate Manager (ACM) and integrated into the ALB listener on port 443.

Steps:

- i. Request SSL Certificate**
 - **Service:** AWS Certificate Manager (ACM)
 - **Domain Names:**
 - `www.frontend.gcpclouds.life`

- *.gcpclouds.life
- **Validation Method:** DNS (Recommended)
- **Route 53:** Add the required CNAME records for DNS validation
- Status should become Issued

The screenshot shows the AWS Certificate Manager console for a certificate with identifier **e6925b04-5562-4bbc-90fe-6595ec7bd806**. The certificate status is **Issued**. Below the status, there is a table of domains with 2 entries:

Domain	Status	Renewal status	Type	CNAME name	CNAME value
frontend.gcpclouds.life	Success	-	CNAME	_75bb030a7858e13ac20d100362af337f.frontend.gcpclouds.life.	_1468c70186f612c393e3a143785d1e1.xlfgmrvlj.acm-validations.aws.
*.gcpclouds.life	Success	-	CNAME	_e3398eac1285c7aa2026a98ed618822d.gcpclouds.life.	_3f2f397fe2aaa5510a68b7e173eb1981.xlfgmrvlj.acm-validations.aws.

The details section shows the certificate is in use, with a serial number, requested at date, and renewal eligibility.

ii. Configure ALB Listeners

- Port 443 → HTTPS
 - Add rule to forward to target group (frontend-tg/backend-tg)
 - Attach ACM Certificate to the listener
- Port 80 → Add rule to redirect to HTTPS (Optional but best practice)

The screenshot shows the AWS Elastic Load Balancing console for a target group named **frontend-tg**. The details section shows the target type is **Instance**, the protocol is **HTTP**, and the port is **80**. The target group is associated with the **front-end-alb** load balancer. The target group has 1 total target, which is **Healthy**. The distribution of targets by Availability Zone (AZ) shows 1 target in the **ap-south-1b** AZ.

The registered targets table shows the following target:

Instance ID	Name	Port	Zone	Health status	Health status details	Admin...	Overri...	Launch...	Anom...
i-038e98ebab61a6045	frontend	80	ap-south-1b (a...	Healthy	-	No override	No overri...	July 7, 20...	Ne...

ap-south-1.console.aws.amazon.com/ec2/home?region=ap-south-1#LoadBalancer:loadBalancerArn=arn:aws:elasticloadbalancing:ap-south-1:767828764671:loadbalancer/app/gcpcldouds-front...

Search [Alt+S]

Asia Pacific (Mumbai) Prabath D

EC2 > Load balancers > gcpcldouds-frontend-alb

Details

Load balancer type
Application

Scheme
Internet-facing

Status
Active

Hosted zone
ZP97RAFLXTNZK

VPC
vpc-07c7956f922556a60

Availability Zones
subnet-0cbd76bd2fa91f51c ap-south-1c (aps1-az2)
subnet-00bdabdd2ad0e63a ap-south-1a (aps1-az1)
subnet-084a18a2c6837df33 ap-south-1b (aps1-az3)

Load balancer IP address type
IPv4

Date created
July 7, 2025, 15:34 (UTC+05:30)

Load balancer ARN
arn:aws:elasticloadbalancing:ap-south-1:767828764671:loadbalancer/app/gcpcldouds-frontend-alb/596cbd5a089ef628

DNS name
gcpcldouds-frontend-alb-384370661.ap-south-1.elb.amazonaws.com (A Record)

Listeners and rules | Network mapping | Resource map | Security | Monitoring | Integrations | Attributes | Capacity | Tags

Listeners and rules (2)

A listener checks for connection requests on its configured protocol and port. Traffic received by the listener is routed according to the default action and any additional rules.

Filter listeners

< 1 >

Protocol:Port	Default action	Rules	ARN	Security policy	Default SSL/TLS certificate	mTLS
☐ HTTP:80	Forward to target group frontend-tg 1 (100%) Target group stickiness: Off	1 rule	ARN	Not applicable	Not applicable	Not applicable

CloudShell Feedback © 2025, Amazon Web Services, Inc. or its affiliates. Privacy Terms Cookie preferences

ap-south-1.console.aws.amazon.com/ec2/home?region=ap-south-1#LoadBalancer:loadBalancerArn=arn:aws:elasticloadbalancing:ap-south-1:767828764671:loadbalancer/app/gcpcldouds-front...

Search [Alt+S]

Asia Pacific (Mumbai) Prabath D

EC2 > Load balancers > gcpcldouds-frontend-alb

VPC
vpc-07c7956f922556a60

Availability Zones
subnet-00bdabdd2ad0e63a ap-south-1a (aps1-az1)
subnet-084a18a2c6837df33 ap-south-1b (aps1-az3)

Load balancer IP address type
IPv4

Date created
July 7, 2025, 15:34 (UTC+05:30)

Load balancer ARN
arn:aws:elasticloadbalancing:ap-south-1:767828764671:loadbalancer/app/gcpcldouds-frontend-alb/596cbd5a089ef628

DNS name
gcpcldouds-frontend-alb-384370661.ap-south-1.elb.amazonaws.com (A Record)

Listeners and rules | Network mapping | Resource map | Security | Monitoring | Integrations | Attributes | Capacity | Tags

Listeners and rules (2)

A listener checks for connection requests on its configured protocol and port. Traffic received by the listener is routed according to the default action and any additional rules.

Filter listeners

< 1 >

Protocol:Port	Default action	Rules	ARN	Security policy	Default SSL/TLS certificate	mTLS
☐ HTTP:80	Forward to target group frontend-tg 1 (100%) Target group stickiness: Off	1 rule	ARN	Not applicable	Not applicable	Not applicable
☐ HTTPS:443	Forward to target group frontend-tg 1 (100%) Target group stickiness: Off	1 rule	ARN	ELBSecurityPolicy-TLS13-1-2-...	frontend.gcpcldouds.life (Certifi...	Off

CloudShell Feedback © 2025, Amazon Web Services, Inc. or its affiliates. Privacy Terms Cookie preferences

iii. Security Groups

- Ensure inbound ports 80 and 443 are open
- Remove public IP access if NAT Gateway or Bastion is used.
- After the configuration of security groups attach it to the respective services to have access with the protection.

The screenshot displays the AWS Management Console interface for a Security Group. The breadcrumb navigation shows the path: EC2 > Security Groups > sg-0a120cc6d70b3cf6d - all_tcp. The main content area is titled 'sg-0a120cc6d70b3cf6d - all_tcp' and includes an 'Actions' dropdown menu.

Details

Security group name all_tcp	Security group ID sg-0a120cc6d70b3cf6d	Description all_tcp	VPC ID vpc-07c7956f922556a60
Owner 767828764671	Inbound rules count 5 Permission entries	Outbound rules count 1 Permission entry	

Inbound rules | Outbound rules | Sharing - new | VPC associations - new | Tags

Inbound rules (5)

	Name	Security group rule ID	IP version	Type	Protocol	Port range	Source	Description
☐	-	sgr-03e14b9ef7301a4e9	IPv4	SSH	TCP	22	0.0.0.0/0	-
☐	-	sgr-00e0002b1490f3404	IPv4	All TCP	TCP	0 - 65535	0.0.0.0/0	-
☐	-	sgr-0c789a4060089b096	IPv4	HTTP	TCP	80	0.0.0.0/0	-
☐	-	sgr-0f48431c47cf7c8db	IPv4	HTTPS	TCP	443	0.0.0.0/0	-
☐	-	sgr-00075391d762b3992	-	MYSQL/Aurora	TCP	3306	sg-0a120cc6d70b3cf6d...	-

CloudShell | Feedback | © 2025, Amazon Web Services, Inc. or its affiliates. | Privacy | Terms | Cookie preferences

iv. Validation

- Access <https://frontend.gcpclouds.life> and <https://api.gcpclouds.life>
- Confirm padlock, certificate validity, and page load over HTTPS
- Validate everything and proceed with the registering the students and store the data in the MySQL.

us-east-1.console.aws.amazon.com/route53/v2/hostedzones?region=ap-south-1#ListRecordSets/Z04902763HQ9T6VR3KUZA

gcpclouds.life

Public gcpclouds.life Info

Delete zoneTest recordConfigure query logging

Hosted zone detailsEdit hosted zone

Records (5)DNSSEC signingHosted zone tags (0)

Records (5) Info

Automatic mode is the current search behavior optimized for best filter results. To change modes go to settings.

Filter records by property or valueTypeRouting p...Alias

Record name	Type	Routin...	Differ...	Alias	Value/Route traffic to	TTL (\$...	Health ...	Evalua...	Record ID
gcpclouds.life	NS	Simple	-	No	ns-118.awsdns-14.com. ns-601.awsdns-11.net. ns-1156.awsdns-16.org. ns-1830.awsdns-36.co.uk.	172800	-	-	-
gcpclouds.life	SOA	Simple	-	No	ns-118.awsdns-14.com. awsd...	900	-	-	-
_e3398eac1285c7aa202...	CNAME	Simple	-	No	_3f2f397fe2aaa5510a68b7e...	300	-	-	-
frontend.gcpclouds.life	A	Simple	-	Yes	dualstack.gcpclouds-fronten...	-	-	Yes	-
_75bb030a7858e13ac20...	CNAME	Simple	-	No	_1468c70186f6f12c393e3a1...	300	-	-	-

CloudShellFeedback

© 2025, Amazon Web Services, Inc. or its affiliates. PrivacyTermsCookie preferences

frontend.gcpclouds.life

frontend.gcpclouds.life

Connection is secure

Cookies and site data

Site settings

About this page

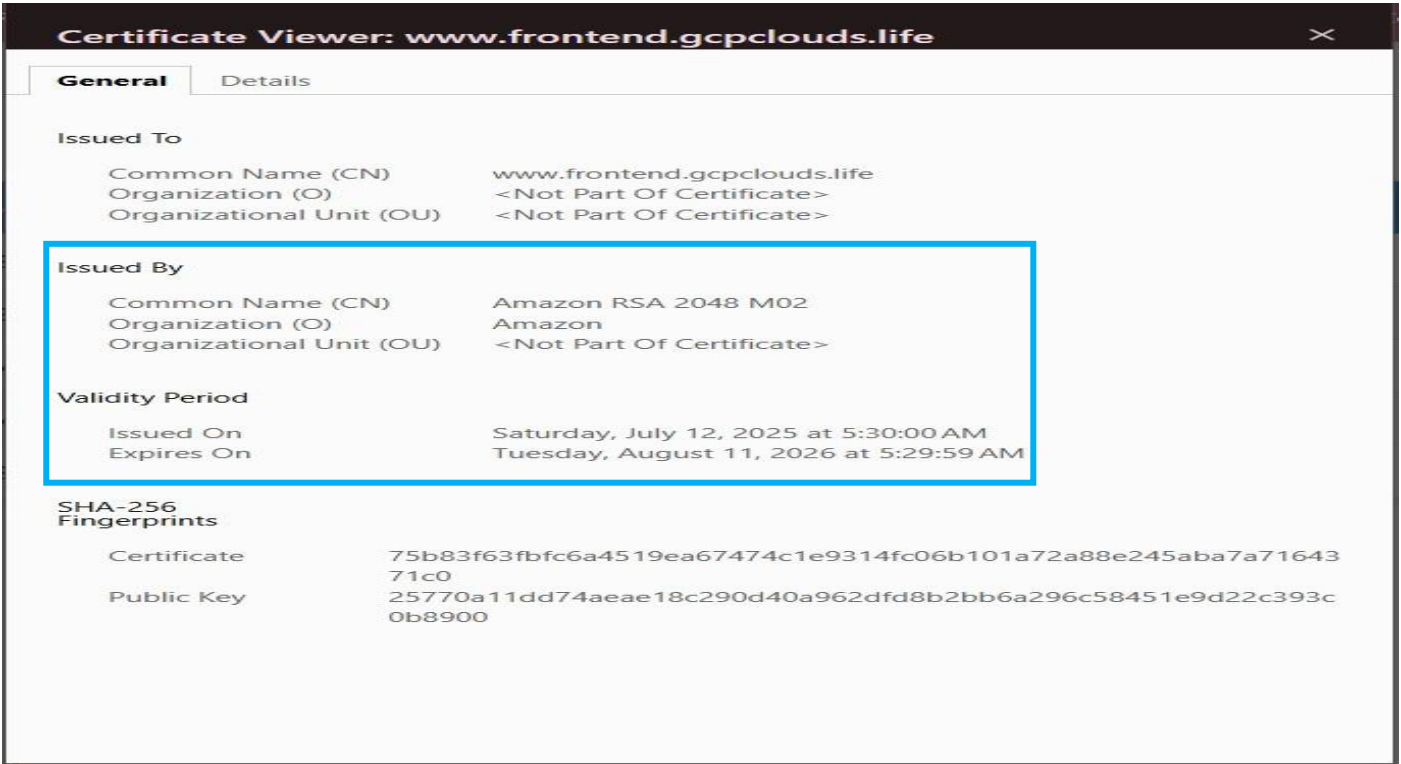
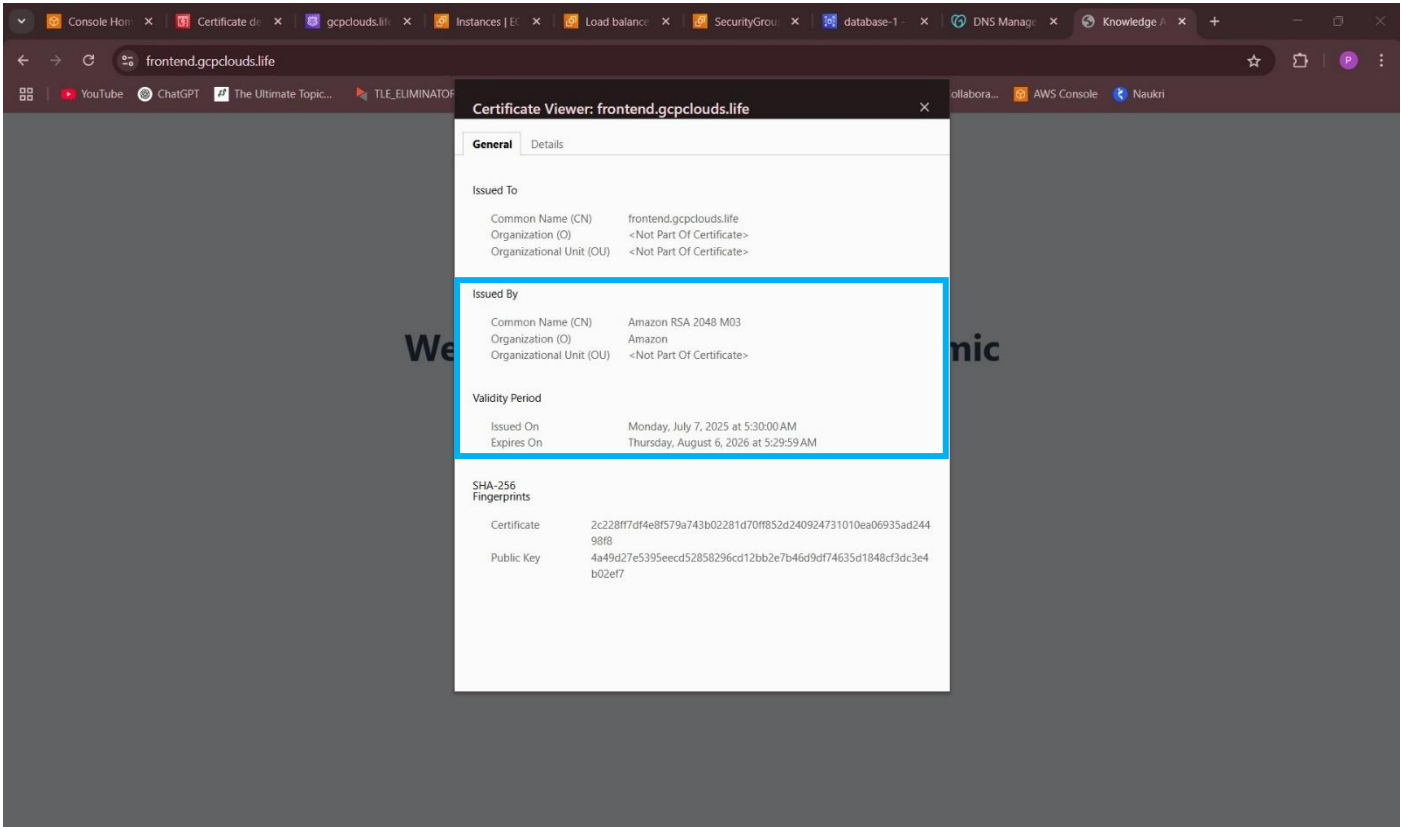
Welcome to Greens Tech Academic

Sign InSign Up

Username

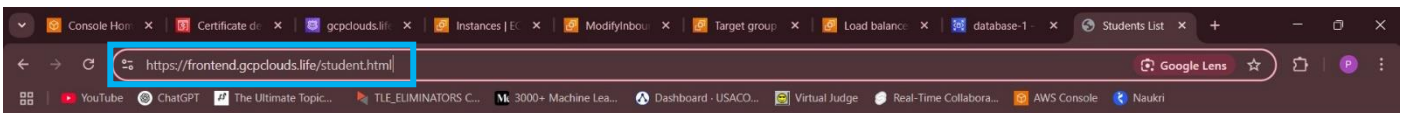
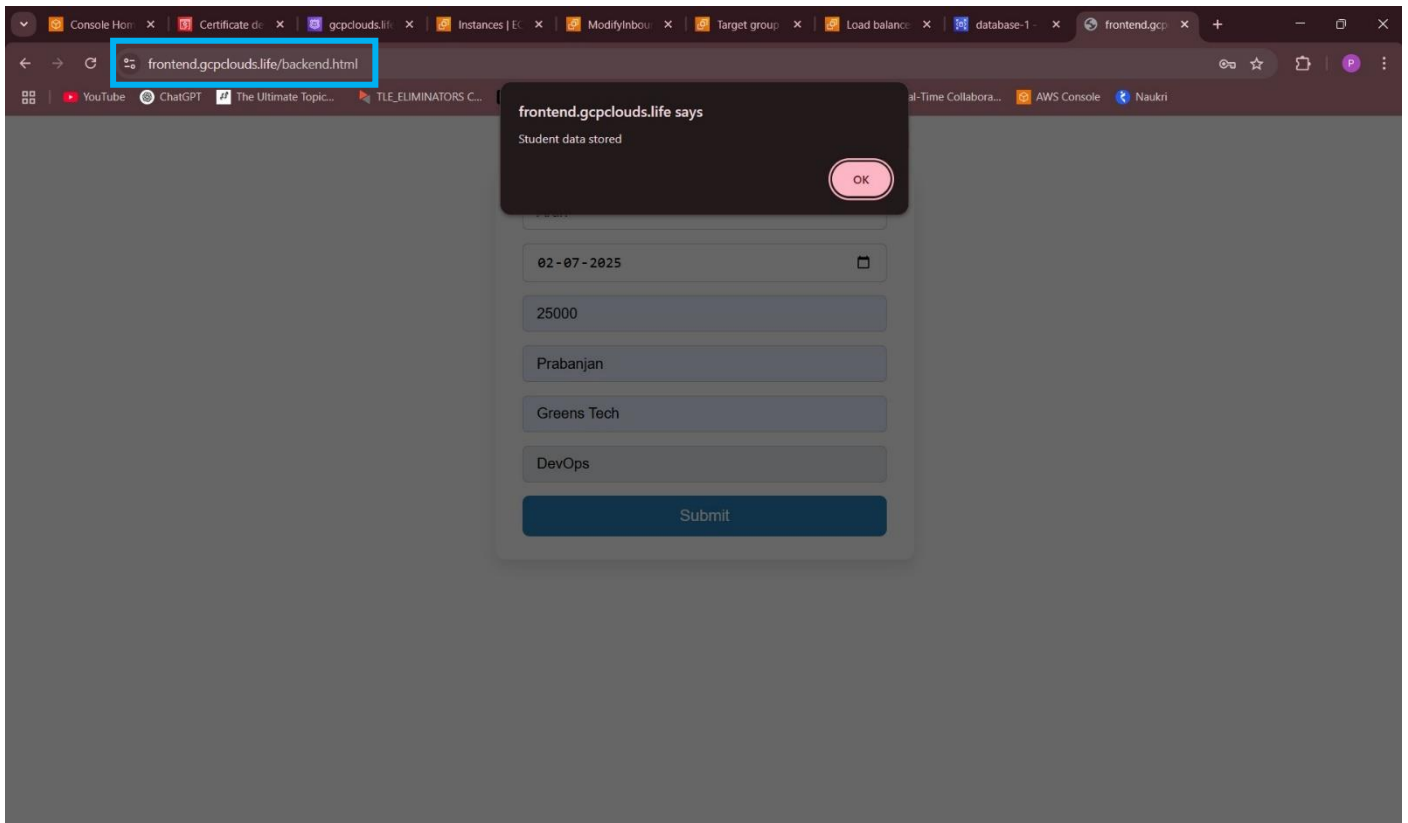
Password

Sign In



v. Student Data:

- Register the student's data and validate the output.



Students Registered

Name	Date of Joining	Fees Paid	Department	Trainer Name	Company Name
Prabath	7/7/2025	23000.00	DevOps	Prabanjan	Greens Tech
Arun	7/2/2025	25000.00	DevOps	Prabanjan	Greens Tech
Neeraj	6/30/2025	30000.00	DevOps	Venkat	Greens Tech
Darius Rijin	7/11/2025	26000.00	DevOps	Pavan	Greens Tech
Jayanthan	7/4/2025	20000.00	DevOps	Mubarak	Greens Tech

➤ Henceforth, the task to enable HTTPS using ACM and ALB Listener Rules is completed.

Task 3: Configure IAM, CloudWatch Monitoring, Secrets Manager

Objective:

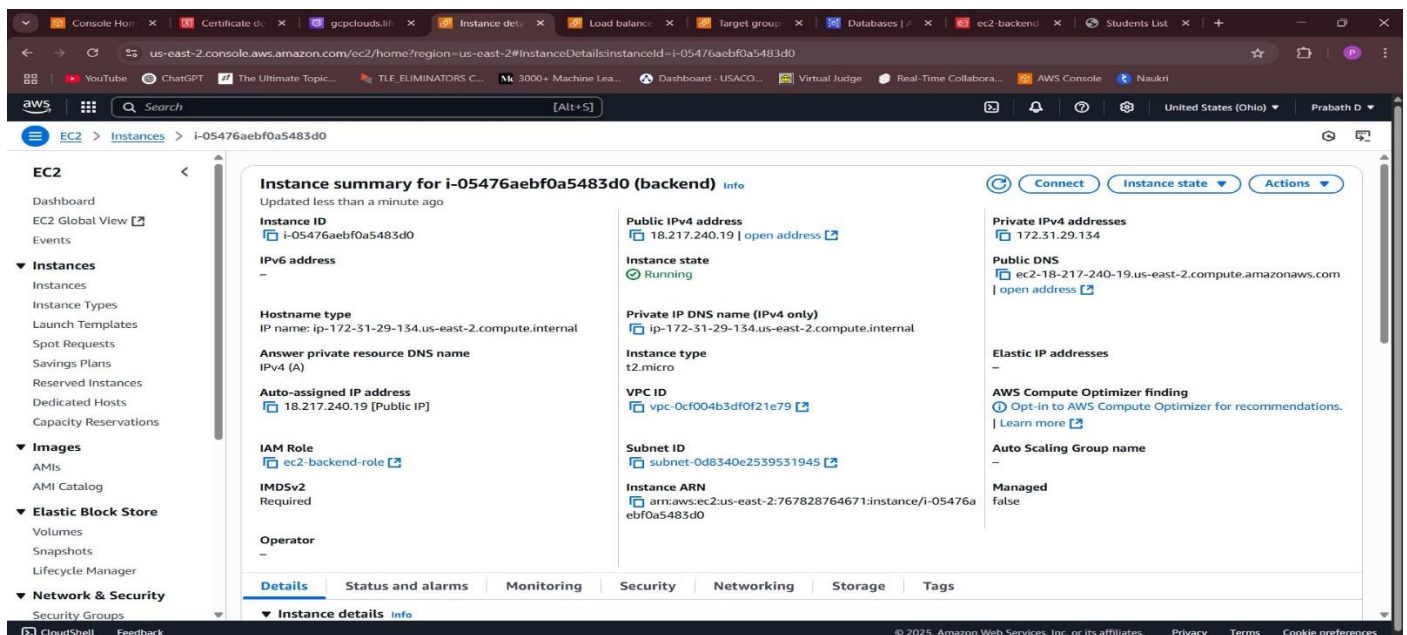
To enhance security, monitoring, and credentials management of the backend application using:

- IAM Role (for secure permission access),
- AWS Secrets Manager (for securely storing DB credentials), and
- Amazon CloudWatch (for monitoring resource metrics and triggering alerts).

1. IAM Role for EC2 (Backend)

Steps:

- Go to IAM > Roles > Create Role.
- Trusted Entity: AWS Service → EC2
- Attach Permissions Policies:
 - SecretsManagerReadWrite
 - CloudWatchAgentServerPolicy
- Name the Role: backend-ec2-role
- Go to EC2 > Instances > backend-ec2-instance > Actions > Security > Modify IAM Role
 - Attach backend-ec2-role



us-east-2.console.aws.amazon.com/ec2/home?region=us-east-2#InstanceDetailsinstanceId=i-05476aebf0a5483d0

EC2 > Instances > i-05476aebf0a5483d0

EC2

- Dashboard
- EC2 Global View
- Events
- Instances**
 - Instances
 - Instance Types
 - Launch Templates
 - Spot Requests
 - Savings Plans
 - Reserved Instances
 - Dedicated Hosts
 - Capacity Reservations
- Images**
 - AMIs
 - AMI Catalog
- Elastic Block Store**
 - Volumes
 - Snapshots
 - Lifecycle Manager
- Network & Security**
 - Security Groups

Instance Details

Hostname type
IP name: ip-172-31-29-134.us-east-2.compute.internal

Answer private resource DNS name
IPv4 (A)
18.217.240.19 [Public IP]

Auto-assigned IP address
18.217.240.19 [Public IP]

IAM Role
ec2-backend-role

IMDSv2
Required

Operator
-

Private IP DNS name (IPv4 only)
ip-172-31-29-134.us-east-2.compute.internal

Instance type
t2.micro

VPC ID
vpc-0cf004b3df0f21e79

Subnet ID
subnet-0d8340e2539531945

Instance ARN
arn:aws:ec2:us-east-2:767828764671:instance/i-05476aebf0a5483d0

Elastic IP addresses
-

AWS Compute Optimizer finding
Opt-in to AWS Compute Optimizer for recommendations.
Learn more

Auto Scaling Group name
-

Managed
false

Details | **Status and alarms** | **Monitoring** | **Security** | **Networking** | **Storage** | **Tags**

Security details

IAM Role
ec2-backend-role

Security groups
sg-0b8f116c36581c297 (all_tcp)

Owner ID
767828764671

Launch time
Sat Jul 12 2025 19:25:43 GMT+0530 (India Standard Time)

CloudShell Feedback

us-east-1.console.aws.amazon.com/iam/home?region=us-east-2#/roles/details/ec2-backend-role?section=permissions

IAM > Roles > ec2-backend-role

ec2-backend-role [Info](#) [Delete](#)

Role for EC2 to use Secrets Manager and CloudWatch

Summary [Edit](#)

Creation date
July 12, 2025, 22:22 (UTC+05:30)

ARN
arn:aws:iam::767828764671:role/ec2-backend-role

Instance profile ARN
arn:aws:iam::767828764671:instance-profile/ec2-backend-role

Last activity
-

Maximum session duration
1 hour

Permissions | **Trust relationships** | **Tags** | **Last Accessed** | **Revoke sessions**

Permissions policies (3) [Info](#) [Simulate](#) [Remove](#) [Add permissions](#)

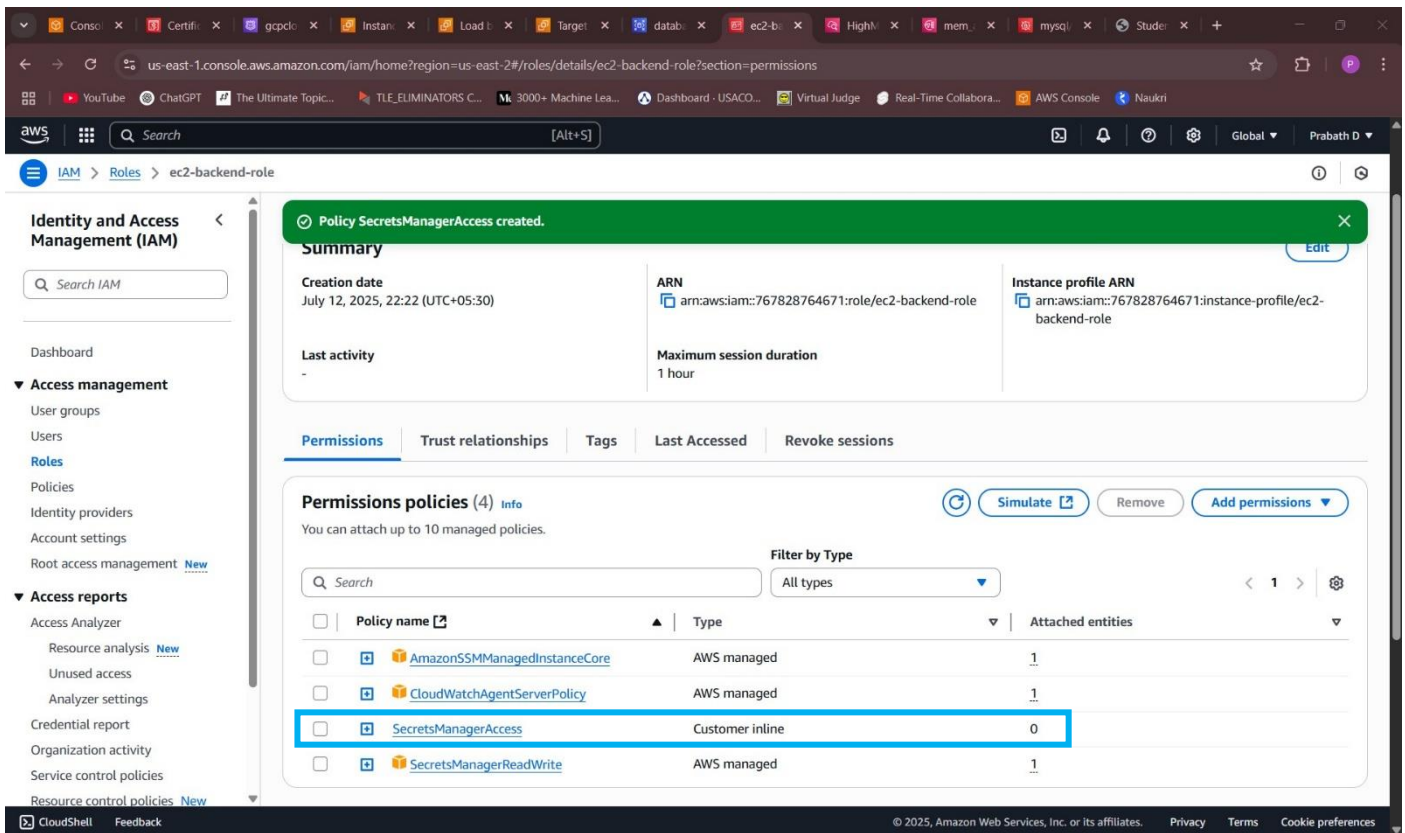
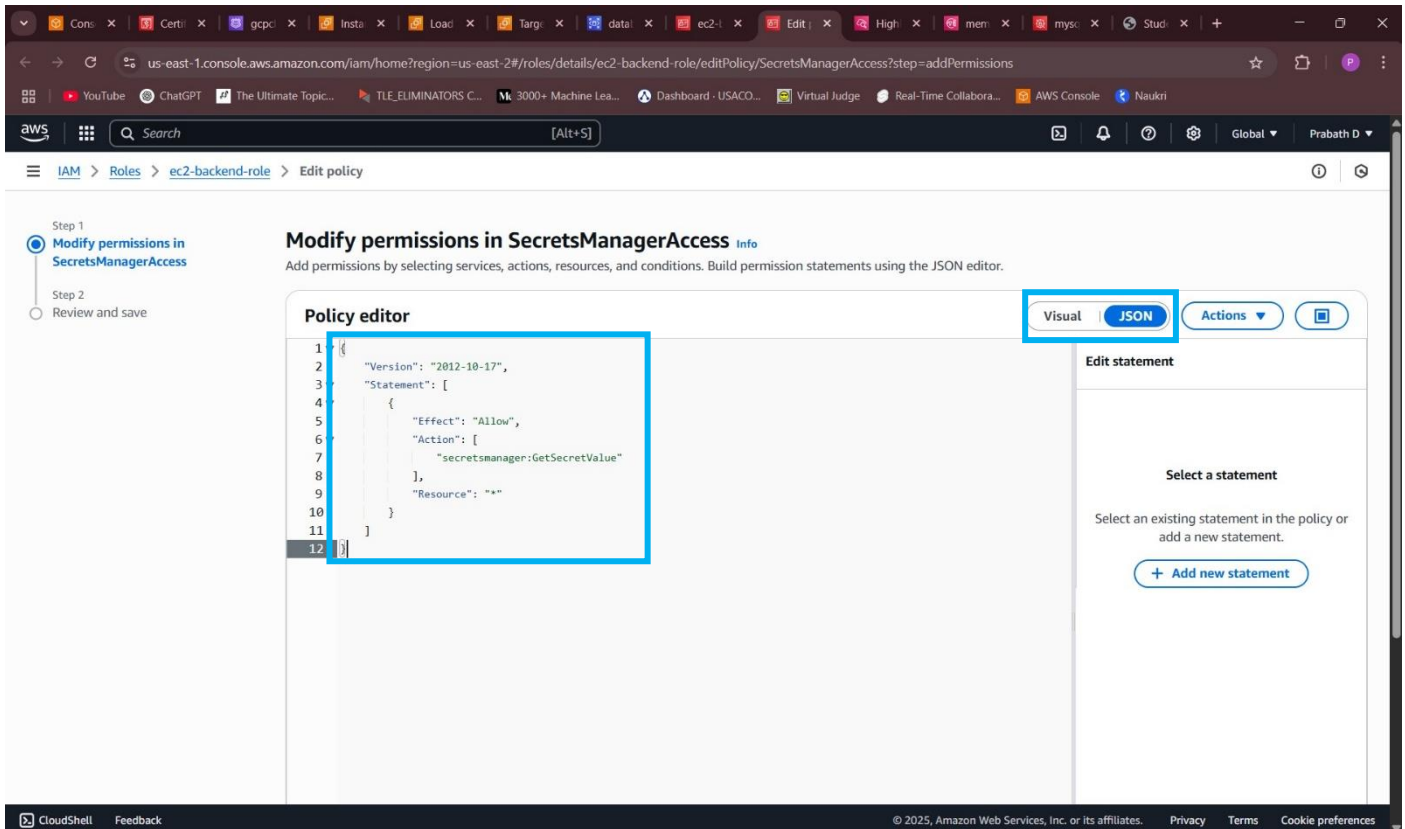
You can attach up to 10 managed policies.

Filter by Type
All types

Policy name | **Type** | **Attached entities**

☐ AmazonSSMManagedInstanceCore	AWS managed	1
☐ CloudWatchAgentServerPolicy	AWS managed	1
☐ SecretsManagerReadWrite	AWS managed	1

CloudShell Feedback



Added Custom Inline Policy(JSON Code)

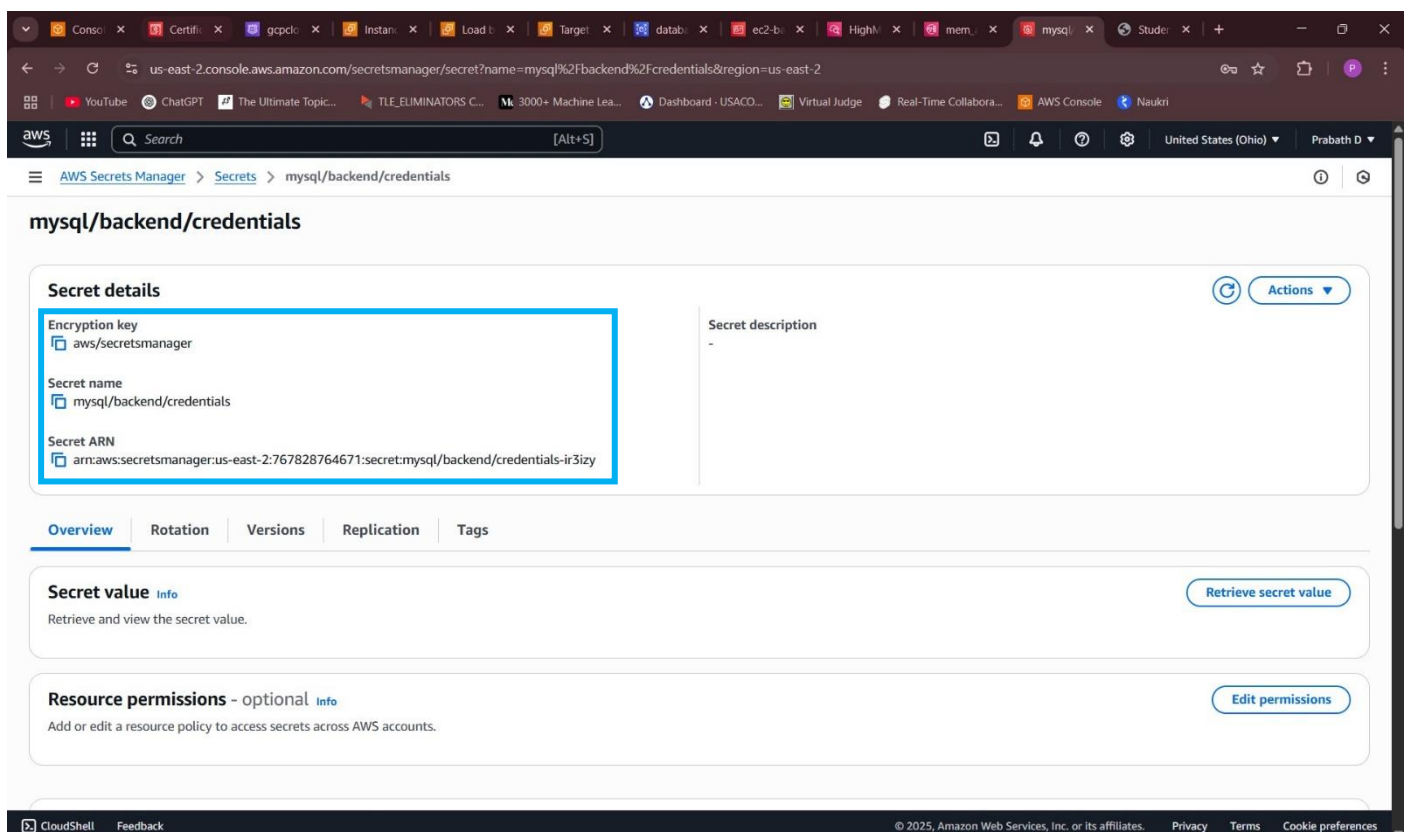
Outcome:

The backend instance can securely access Secrets Manager and send metrics to CloudWatch.

2. Store DB Credentials in Secrets Manager

Steps:

- i. Go to Secrets Manager > Store a new secret.
- ii. Select Credentials for RDS database.
- iii. Enter your:
 - Username: admin
 - Password: YourSecurePassword
- iv. Choose Other type of database → Provide DB endpoint, port, and name.
- v. Secret Name: rds/db-credentials
- vi. Store the secret and note the ARN.



Outcome:

Your application will fetch DB credentials securely from Secrets Manager.

3. Update Flask App to Fetch Secrets:

- Here's how your app.py connects securely:

Python:

```
import boto3
```

```
import json
```

```
def get_db_credentials():
```

```
    secret_name = "rds/db-credentials"
```

```
    region_name = "us-east-2"
```

```
    client = boto3.client("secretsmanager", region_name=region_name)
```

```
    response = client.get_secret_value(SecretId=secret_name)
```

```
    secret = json.loads(response["SecretString"])
```

```
    return secret
```

```
secrets = get_db_credentials()
```

```
conn = mysql.connector.connect(
```

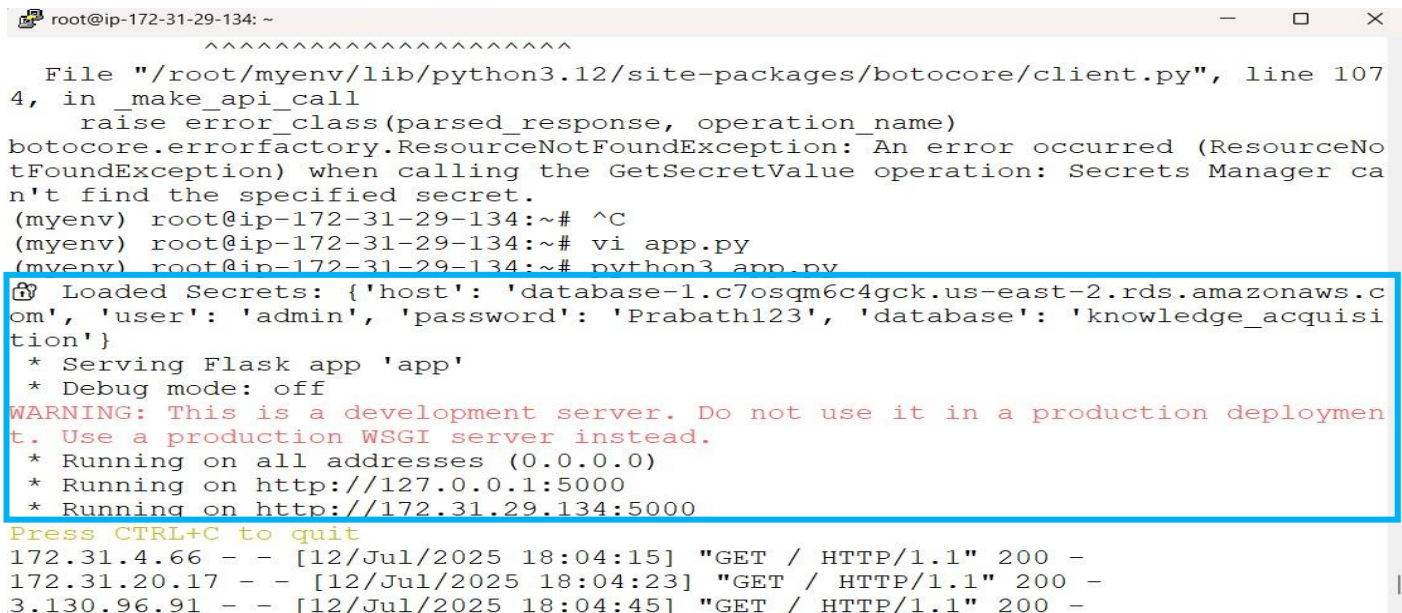
```
    host=secrets["host"],
```

```
    user=secrets["username"],
```

```
    password=secrets["password"],
```

```
    database=secrets["dbname"]
```

```
)
```



The image shows a terminal window with the following content:

```
root@ip-172-31-29-134: ~  
^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^  
File "/root/myenv/lib/python3.12/site-packages/botocore/client.py", line 107  
4, in _make_api_call  
    raise error_class(parsed_response, operation_name)  
botocore.errorfactory.ResourceNotFoundException: An error occurred (ResourceNo  
tFoundException) when calling the GetSecretValue operation: Secrets Manager ca  
n't find the specified secret.  
(myenv) root@ip-172-31-29-134:~# ^C  
(myenv) root@ip-172-31-29-134:~# vi app.py  
(myenv) root@ip-172-31-29-134:~# python3 app.py  
* Loaded Secrets: {'host': 'database-1.c7osqm6c4gck.us-east-2.rds.amazonaws.c  
om', 'user': 'admin', 'password': 'Prabath123', 'database': 'knowledge_acquisi  
tion'}  
* Serving Flask app 'app'  
* Debug mode: off  
WARNING: This is a development server. Do not use it in a production deploymen  
t. Use a production WSGI server instead.  
* Running on all addresses (0.0.0.0)  
* Running on http://127.0.0.1:5000  
* Running on http://172.31.29.134:5000  
Press CTRL+C to quit  
172.31.4.66 - - [12/Jul/2025 18:04:15] "GET / HTTP/1.1" 200 -  
172.31.20.17 - - [12/Jul/2025 18:04:23] "GET / HTTP/1.1" 200 -  
3.130.96.91 - - [12/Jul/2025 18:04:45] "GET / HTTP/1.1" 200 -
```

4. Install and Configure CloudWatch Agent on EC2

Steps:

A. Install CloudWatch Agent:

```
sudo apt update
```

```
sudo apt install amazon-cloudwatch-agent -y
```

B. Configure CloudWatch Agent:

Create file `/opt/aws/amazon-cloudwatch-agent/bin/config.json` with:

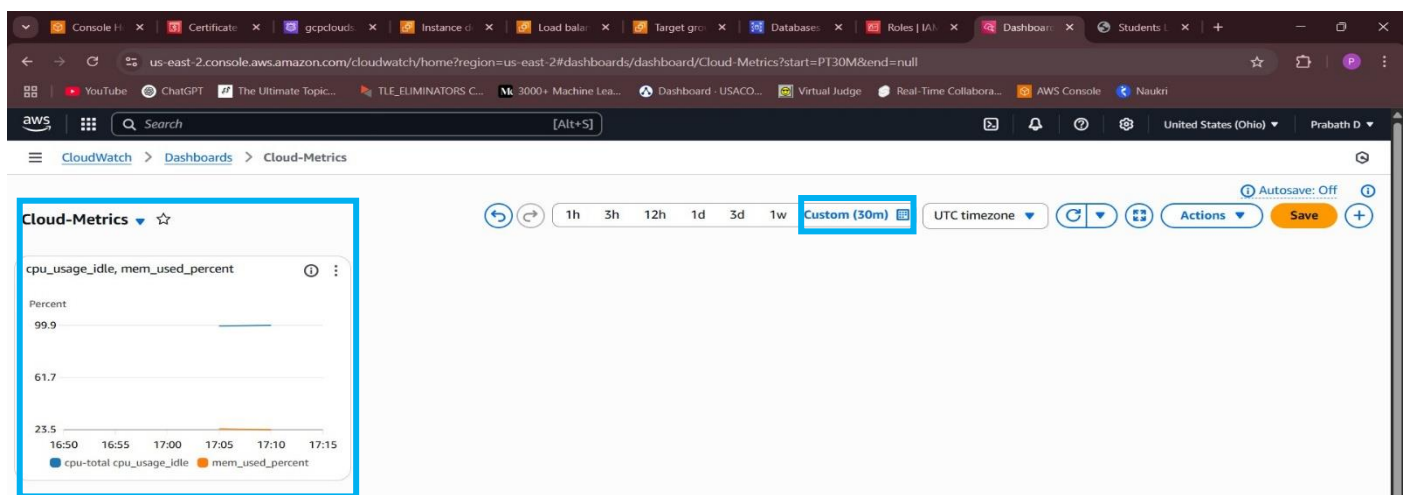
JSON:

```
{
  "metrics": {
    "metrics_collected": {
      "mem": { "measurement": ["mem_used_percent"], "metrics_collection_interval":
60 },
      "cpu": { "measurement": ["cpu_usage_idle"], "metrics_collection_interval": 60 }
    }
  }
}
```

C. Start Agent:

```
sudo /opt/aws/amazon-cloudwatch-agent/bin/amazon-cloudwatch-agent-ctl \
-a fetch-config \
-m ec2 \
-c file:/opt/aws/amazon-cloudwatch-agent/bin/config.json \
-s
```

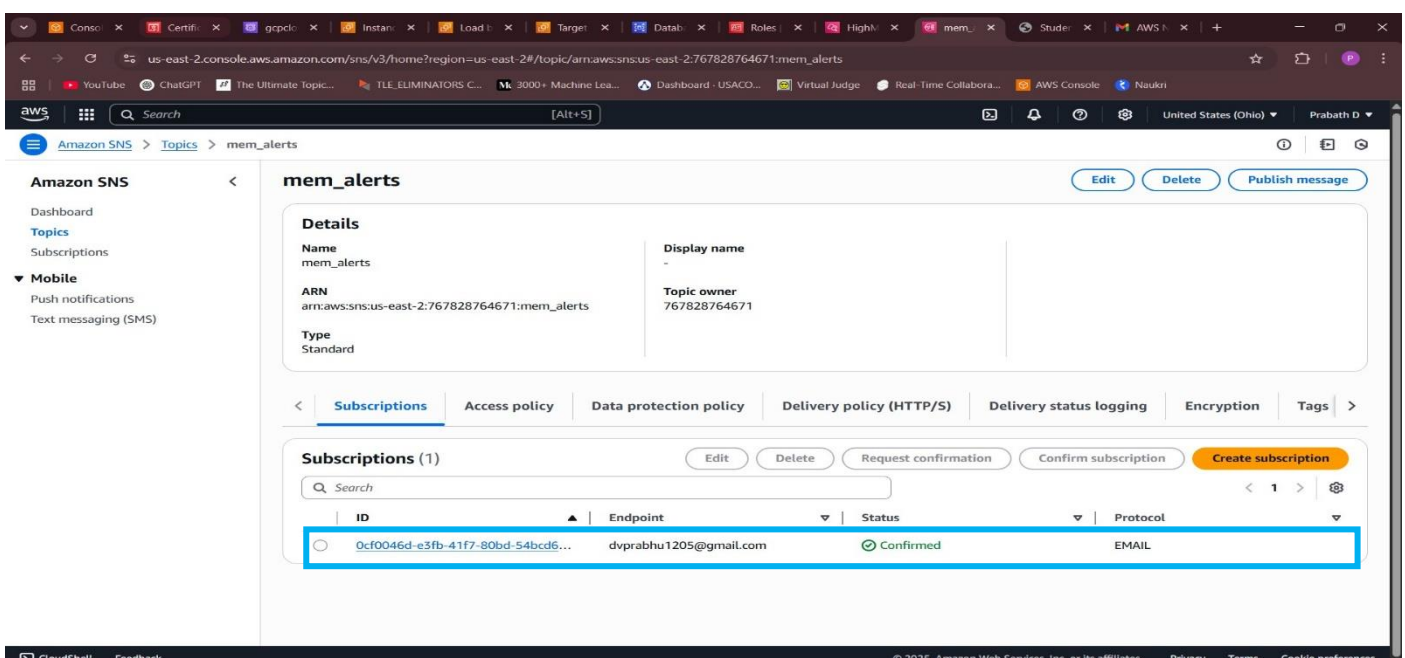
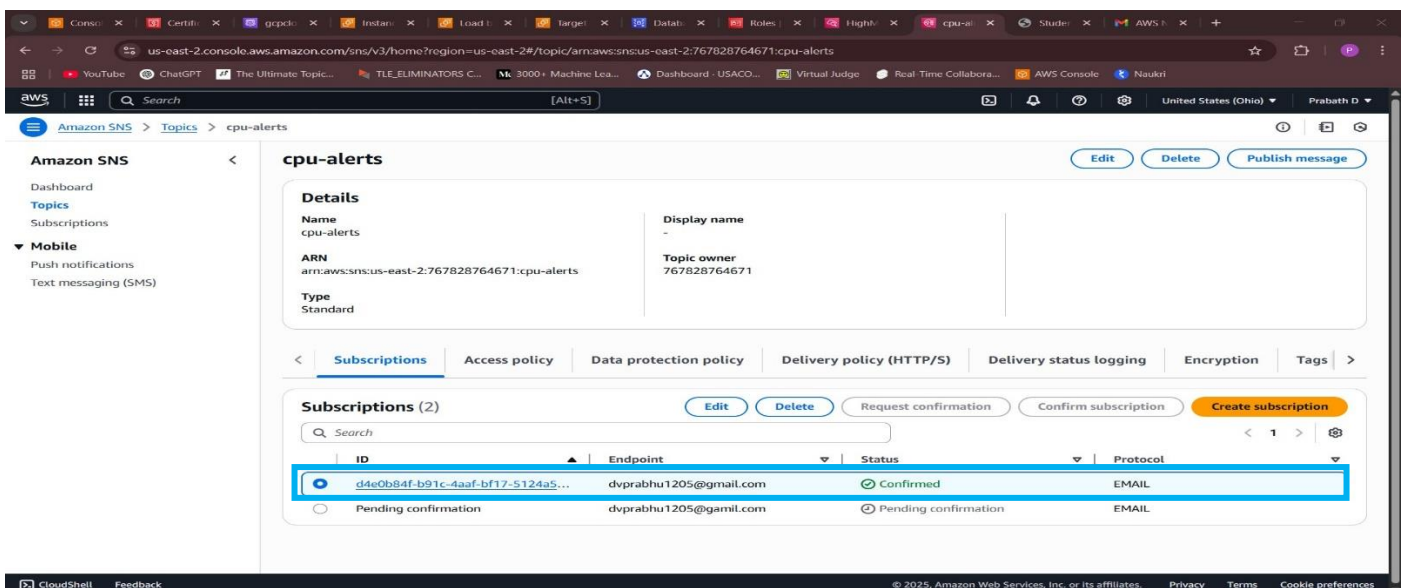
Outcome: Memory and CPU metrics will appear in CloudWatch Metrics > CWAgent namespace



5. Create CloudWatch Alarm + SNS Alert

Steps:

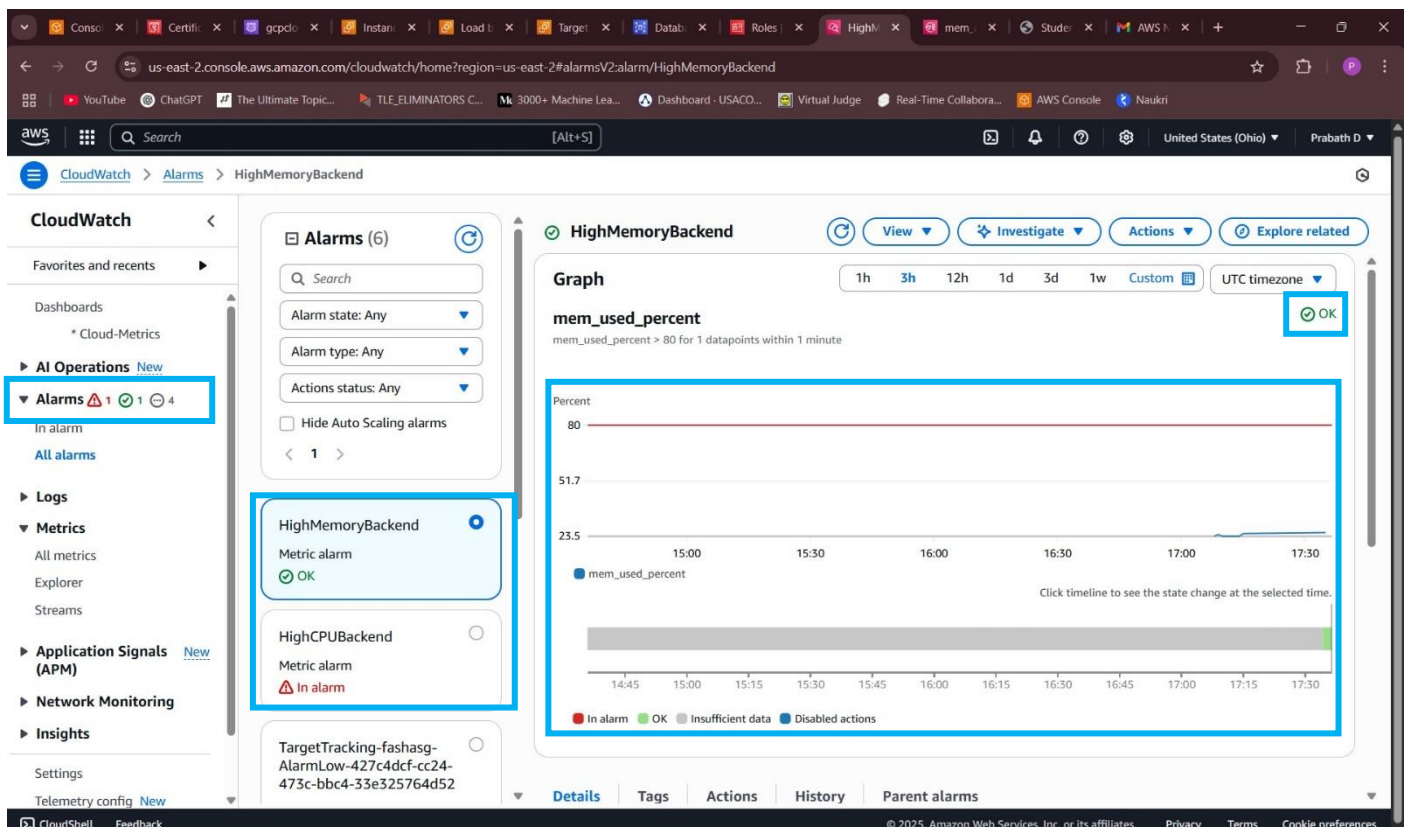
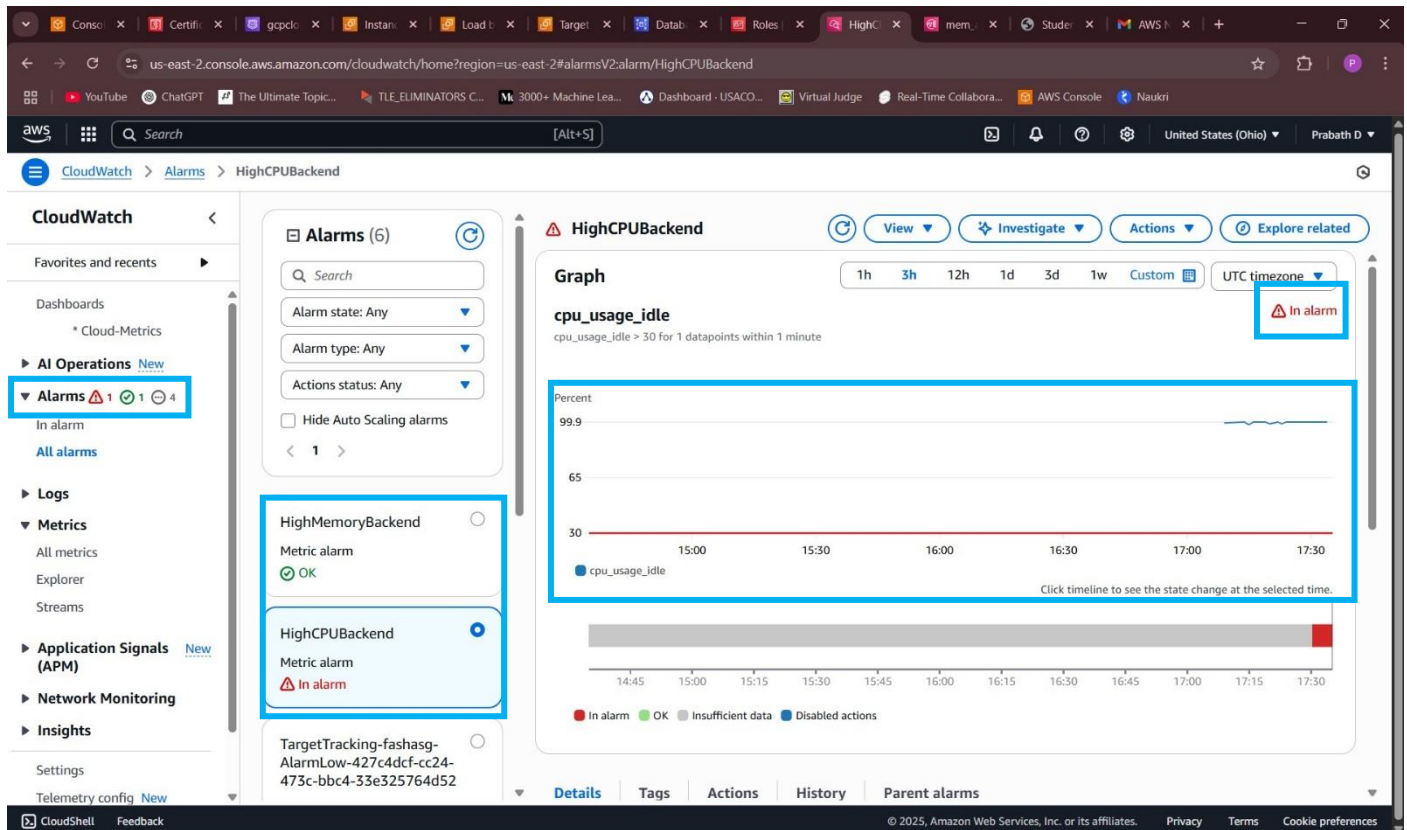
1. Go to CloudWatch > Alarms > Create Alarm.
2. Select CWAgent > mem_used_percent
3. Set Threshold (e.g., $\geq 70\%$).
4. Actions:
 - Create new SNS Topic (e.g., alert-topic)
 - Add your email and confirm subscription
5. Review and create



Outcome: Receives email alerts if memory or CPU crosses threshold.

6. Alert Validation:

- Shows two alerts with activity status of CPU and Memory.



➤ Henceforth, the task to Configure IAM, CloudWatch Monitoring, Secrets Manager is completed.

Task 4: Implement a Cloud Craft 3D Diagram for this Project

Objective

This project implements a secure and monitored 3-tier web application hosted on AWS using:

- Frontend (Apache2, HTML/JS)
- Backend (Flask + Python + MySQL)
- Database (Amazon Aurora/MySQL)

It includes HTTPS support, domain mapping via Route 53, IAM roles, Secrets Manager, and CloudWatch monitoring.

I. Architecture Layers:

i. Route 53 (Outside VPC)

- Manages custom domain: frontend.gcpclouds.life and api.gcpclouds.life.
- Implements DNS routing to ALBs (HTTPS).

II. Component Purposes:

Component	Purpose
Route 53	Maps custom domain names to ALBs
Application Load Balancer	Distributes traffic over EC2 instances (frontend + backend)
Frontend EC2	Serves static content (HTML, CSS, JS)
Backend EC2	Flask REST API to process signup/login data
Secrets Manager	Stores DB credentials securely
IAM Role	Grants EC2 access to Secrets Manager + CloudWatch
CloudWatch Agent	Monitors CPU/Memory of EC2, collects metrics
SNS Alarm	Sends email alerts when resource thresholds are breached
Amazon RDS (Aurora)	Stores user and student data

III. Additional Notes

- IAM role is attached only to the backend EC2 instance.
- CloudWatch monitors CPU and memory and triggers SNS alarms.
- Secrets Manager stores DB credentials and is retrieved inside app.py.
- HTTPS is enabled using ACM certificates on both ALBs (frontend & backend).

IV. Tagging & Visual Enhancements in CloudCraft

- Use labels to annotate EC2 instances with software (e.g., Apache, Flask).
- Use light boxes or colored overlays to separate:
 - Frontend Tier
 - Backend Tier
 - Database Tier
- All resources are placed inside a Default VPC box for clarity.

V. Validation

- Access via:
 - <https://frontend.gcpclouds.life>
 - <https://api.gcpclouds.life>
- Confirm alerts from CloudWatch when memory/cpu usage goes high.
- Validate backend API calling Secrets Manager without hardcoded passwords.

